HAAB AVIATION CONSULTANCY SERVICES

Training Management Platform

Product and Technical Specification

Product requirements, technical requirements, application flow, UI/UX design brief and backend schema, in one document.

Document	Details
Product	HAAB Training Management Platform (private, invite-only)
Contents	1 Product requirements · 2 Technical requirements · 3 Application flow · 4 UI/UX design brief · 5 Backend schema
Version	1.0, October 2026. Describes the platform as built and deployed to staging.
Prepared by	Anknovate IT, for HAAB Aviation Consultancy Services
Companion	The separate User Handbook explains every screen and task for each role.

Contents

How to read this document 4

Part 1. Product Requirements Document 5

1.1 Purpose and background 5
1.2 Vision, goals and non-goals 5
1.3 Users and roles 5
1.4 Scope 6
1.5 Functional requirements 7
1.6 Business rules 9
1.7 Non-functional requirements 9
1.8 Success measures 10
1.9 Assumptions and constraints 10
1.10 Release status and roadmap 10
1.11 Known limits and risks 11

Part 2. Technical Requirements Document 12

2.1 Architecture 12
2.2 Technology stack 13
2.3 Repository layout 13
2.4 Environments and deployment 14
2.5 Authentication and sessions 14
2.6 Authorisation 15
2.7 API design 16
2.8 Data access and migrations 22
2.9 Security controls 22
2.10 Background jobs 23
2.11 Files and storage 23
2.12 Notifications and email 23
2.13 Generated documents 24
2.14 Audit log 24
2.15 Configuration 24
2.16 Settings registry 25
2.17 Testing and quality 25
2.18 Operations 26

Part 3. Application Flow 27

3.1 The whole training cycle 27
3.2 Getting in 27
3.3 Where each role goes 28
3.4 Registering people on a programme 28
3.5 Programme life cycle 29
3.6 Attendance 29
3.7 Assessments and marking 30
3.8 Results, release and close-out 30
3.9 Certificates 30
3.10 Finance 31
3.11 Learning material 31
3.12 Compliance, documents and reports 31
3.13 Settings, deletions and reset 31
3.14 State definitions 32

Part 4. UI/UX Design Brief 33

4.1 Design intent 33
4.2 Brand 33
4.3 Layout and navigation 34
4.4 Component library 36

4.5 Key screens 36
4.6 Accessibility 38
4.7 Content and microcopy 38
4.8 Design QA checklist 38

Part 5. Backend Schema 40

5.1 Overview 40
5.2 People and organisations 41
5.3 Catalogue and learning material 43
5.4 Programmes and delivery 47
5.5 Assessment, results and certificates 50
5.6 Finance 55
5.7 Compliance, documents and platform 57
5.8 Allowed values 60
5.9 Functions, triggers and extensions 61
5.10 Migrations 61
5.11 Retention and deletion 62

How to read this document

This specification describes the platform **as it is built**, not only as it was first intended. Where the original plan was changed on purpose, the change is stated. It is written for four readers.

Reader	Start with	Why
HAAB leadership and product owners	Part 1	What the platform is for, who uses it, what it does and what it deliberately does not do yet.
Engineers and DevOps	Parts 2 and 5	Architecture, security, interfaces, deployment, configuration and the full data model.
Testers and trainers	Part 3	The end-to-end flows and the rules behind each status change.
Designers and front-end developers	Part 4	Brand, layout, navigation, components and the accessibility rules.

> **NAMING**
> The product is called the **HAAB Training Management Platform**. Inside the code and database it is still abbreviated **ATMP** (Aviation Training Management Platform), which is why certificate numbers begin with ATMP.

Part 1. Product Requirements Document

1.1 Purpose and background

HAAB Aviation Consultancy Services Ltd. is a Ghana-based aviation consultancy (incorporated July 2019) that trains staff of civil aviation authorities, airport operators, ground handlers and other aviation organisations across Africa. Its courses follow ICAO standards and recommended practices, ACI and IATA guidance. Before this platform, registers, results, certificates and invoices lived in spreadsheets, paper and email threads.

The **HAAB Training Management Platform** replaces them with one secure system covering the whole training life cycle: catalogue, scheduling, registration, attendance, assessment, results, certificates, invoicing, compliance tracking and audit.

1.2 Vision, goals and non-goals

	Statement
Vision	One trusted record of who HAAB trained, how they performed, what they hold a certificate for and when it expires, available to the right person on any device.
Goal 1: Control	Every programme, registration and result is created and changed through one system with an audit trail nobody can quietly edit.
Goal 2: Credibility	Certificates are numbered, carry a QR code and can be verified by anyone, and are only ever issued from evidence (attendance and assessment).
Goal 3: Client service	Client organisations see their own staff's training, results, certificates and invoices, and nobody else's.
Goal 4: Less admin	Reminders, registration deadlines, programme start, timed exams and result calculation run automatically.
Goal 5: Ready to grow	Organisations carry the cost of their trainees today. The platform can open to the general public later, switched on by a Super Administrator, without rebuilding.
Non-goals (this release)	Online payment gateways, SMS and WhatsApp messages, SCORM or hosted video, handwritten-signature images, single sign-on, AI features and a native mobile app. Each has a place reserved in the design (see 1.10).

1.3 Users and roles

The platform is **invite-only**. Nobody can sign up and use it until HAAB has created or approved their account. There are seven roles. A person has exactly one role.

Role	Who they are	Main jobs	Never
Super Administrator	HAAB owner or system owner	Everything a Training Administrator does, plus users of every role, settings, security, integrations, menu access, public access, deletions and the audit log.	Cannot delete the official record or their own account.

Role	Who they are	Main jobs	Never
Training Administrator	HAAB operations staff	Courses, programmes, registrations, attendance, assessments, results, certificates, clients, compliance, documents.	Cannot record payments, change settings or read the full audit log.
Instructor	HAAB facilitator (employee or associate)	Takes attendance, builds assessments and questions, marks answers, registers trainees from Excel, adds learning material to courses they teach.	Never sees fees or invoices. Cannot finalise results unless a Super Administrator grants that one instructor the right.
Trainee	Learner (a client's employee or, later, a member of the public)	Registers for programmes, reads material, checks in, sits assessments, sees released results, downloads certificates.	Sees only their own records.
Client Administrator	A client organisation's training contact	Registers and manages their own organisation's trainees, sees their results, certificates, invoices, documents and compliance.	Sees nothing of any other organisation. Can create trainee accounts only.
Finance Officer	HAAB accounts staff	Invoices, payments, receipts, refunds, finance reports, payment methods and currency.	Cannot touch training records.
Auditor	Internal or external reviewer	Read-only view of everything including the audit log and compliance.	Cannot change anything.

SEGREGATION OF DUTIES

Finance cannot edit training records, and training staff cannot record payments.

The person who marks an assessment cannot finalise the result they marked (a different person must). Overriding a result needs a written reason.

Only a Super Administrator can change settings or delete records.

1.4 Scope

In scope (built)	Out of scope (planned, not built)
Invite-only accounts, optional two-step sign-in, optional self-registration with approval	Single sign-on with Google or Microsoft
Client organisations and their trainees; one organisation per trainee or none	Online payment gateways (Paystack, Hubtel, Flutterwave). Payments are recorded by Finance.
Course categories, catalogue, modules, learning material, question bank	SMS and WhatsApp messaging (credential slots exist)
Programmes with sessions, classrooms, conflict detection and calendar	Hosted video, SCORM, live classes (Zoom or Teams)
Attendance: manual and rotating QR code	Handwritten-signature images on certificates

In scope (built)	Out of scope (planned, not built)
Assessments: seven question types, automatic and manual marking, timed exams	Editable email templates; Day and month calendar grid refinements
Results with maker-checker control, release to trainees	AI features and external HR or aviation-system integrations
Certificates with QR verification, expiry, renewal and revocation	A native mobile app (the web app works on phones)
Invoices, partial payments, receipts, refunds; Public Access switch	A penetration test and a Ghana Data Protection Act review (to be commissioned)
Compliance tracking, documents, reports with export, global search, notifications, audit log	

1.5 Functional requirements

Each requirement has a reference. **Done** means built and covered by the automated end-to-end tests. **Partly** means built with the gap stated. **Not built** means left out on purpose.

Accounts and access

Ref	Requirement	Status
FR-1	Staff create accounts by role; the person receives an email invitation and sets their own password. Nobody is ever given a password by an administrator.	Done
FR-2	Sign-in with email and password; sessions in httpOnly cookies; automatic sign-out after a period of inactivity; throttling of repeated attempts.	Done
FR-3	Optional two-step sign-in with an authenticator app. A Super Administrator can require it for chosen roles.	Done
FR-4	Forgotten-password flow that never reveals whether an account exists.	Done
FR-5	Optional self-registration: a person requests an account, a HAAB administrator approves or rejects it. Off by default.	Done
FR-6	Seven roles with separate permissions, enforced on the server for every request, not only hidden in the menu.	Done
FR-7	A Super Administrator chooses which menu items each role sees (menu level). Real permissions are fixed in code.	Done
FR-8	A Super Administrator can let an individual instructor finalise and release results.	Done

People, clients and courses

Ref	Requirement	Status
FR-10	Trainee profiles with history, search and filters; national ID numbers encrypted at rest and masked on screen; duplicate-name warning.	Done
FR-11	Client organisations with contacts, trainees, registrations, invoices and compliance; strict isolation between clients.	Done
FR-12	Instructor profiles: employment type, specialties, accreditation body and expiry; an instructor cannot be scheduled after accreditation expires.	Done
FR-13	Register trainees in bulk from an Excel or CSV file (up to 500 rows): preview, report every problem first, then create accounts and invitations.	Done
FR-14	Course categories (their code appears in certificate numbers) and a course catalogue with duration, delivery method, pass mark, minimum attendance, certificate validity, capacity and status (Draft, Active, Archived).	Done

Ref	Requirement	Status
FR-15	Course fee is stored but shown only when a Super Administrator turns on Course Fees. Instructors never see it.	Done
FR-16	Modules and learning material (links and uploaded files) per course; instructors add material to courses they teach and remove only what they added.	Done
FR-17	Question bank by course, topic, difficulty and type; bulk import from a Word template with preview.	Done

Training delivery

Ref	Requirement	Status
FR-20	Programmes: one scheduled run of a course with dates, location, delivery, lead instructor, capacity, fee and optional reservation for one client. Six statuses with enforced transitions.	Done
FR-21	Sessions within a programme with date, time, room and instructor; conflicts for instructor, classroom and programme are refused by the application and again by the database.	Done
FR-22	Calendar of sessions; classroom management with capacity.	Done
FR-23	Registrations with Pending, Confirmed, Waitlisted, Completed, Cancelled and No-show; automatic waiting list; promotion from the waiting list when a place opens.	Done
FR-24	Programmes grouped by client (By client view): the courses and programmes each organisation trains on.	Done
FR-25	Attendance per session: Present, Absent, Late, Excused; manual marking and a rotating QR code the trainee scans with their own phone.	Done
FR-26	Programme close-out checklist that tells staff the single next step: start, complete, mark, recalculate, finalise, release.	Done
FR-27	Role-based "To do" lists on the home page.	Done

Assessment, results and certificates

Ref	Requirement	Status
FR-30	Assessments with seven question types, duration, pass mark, attempts, question shuffling and open and close times; timed, auto-saved, auto-submitted at time-out.	Done
FR-31	Automatic marking of objective questions; manual marking of written answers in a marking queue; marks sealed once the result is finalised.	Done
FR-32	Results computed from evidence (attendance and every assessment) and finalised by a person other than the marker; overrides need a reason; release to trainees is a separate step.	Done
FR-33	Completing a programme settles results even if later session dates have not been reached.	Done
FR-34	Certificates issued from a pass: unique number ATMP-year-CATEGORY-serial, QR code, template, PDF, expiry, renewal, reissue, revocation with reason.	Done
FR-35	Public certificate verification by an unguessable link: shows only Valid, Expiring soon, Expired or Revoked.	Done

Finance, compliance and oversight

Ref	Requirement	Status
FR-40	Invoices raised automatically per registration and manually; tax rate and due period from settings; PDF invoices and receipts.	Done
FR-41	Partial payments, receipts, refunds; payment methods configurable; all six invoice statuses.	Done
FR-42	Public Access: organisations pay now; a Super Administrator can later let individuals sign up, register themselves and pay for their own places.	Done
FR-43	Trainees can be switched to see or not see Finance.	Done
FR-44	Compliance requirements by course, client and aviation role with grace periods; missing, expiring and expired training; attendance shortfalls.	Done

Ref	Requirement	Status
FR-45	Documents with type checked by content, size limit, private storage, confidentiality and role-based access; virus scanning when ClamAV is connected.	Done
FR-46	Nine reports with PDF, Excel and CSV export (exports are audited); dashboards for every role; global search scoped to what the role may see.	Done
FR-47	Notifications: in-app and email, queued, retried and never duplicated; class and certificate-expiry reminders. Every email to a registered person ends with a sign-in link.	Done
FR-48	Audit log of who did what, when and from where, with before and after values. It is append-only and hash-chained so tampering is detected.	Done
FR-49	A Super Administrator can delete anything that was added (users, clients, courses with their programmes, programmes, registrations, rooms, assessments, questions, templates) except the official record. A reset command empties demo data.	Done

1.6 Business rules

Rule	Detail
Official record is never deleted	Issued certificates, finalised results and recorded payments are the evidence HAAB is audited on. A delete that would remove them is refused, and the message says what to do instead (revoke, refund, suspend, archive).
Result decision	A trainee passes when attendance meets the course minimum and every published assessment is passed on the best attempt. The result stays Pending until the programme has ended or been marked Completed.
Maker-checker	The person who marked a trainee's evidence cannot finalise that trainee's result. A Super Administrator can switch the control off in Training Rules.
Programme dates and rules	A programme can open for registration only for an Active course. Once registrations exist, its fee cannot change and capacity cannot fall below the places held. Pass mark, attendance and validity of a course cannot change after programmes have started.
Payment before a place	Optional setting: payment must be received before a place is confirmed, with an override reason when it is not.
Trainee and organisation	A trainee belongs to one client organisation or none. HAAB staff belong to HAAB only. People without an organisation can only register themselves when Public Access is on.
Numbering	Certificates ATMP-YYYY-CAT-nnnnnn, invoices INV-YYYY-nnnnnn, receipts RCT-YYYY-nnnnnn, each from its own sequence so numbers are never reused.
Fees	The amount billed always comes from the programme. Course fee is the default copied into new programmes.

1.7 Non-functional requirements

Area	Requirement
Security	Server-side authorisation on every route; encryption in transit and at rest; sensitive fields and integration keys encrypted with AES-256-GCM; rate limiting; secure headers; upload inspection; tamper-evident audit log. See 2.9.

Area	Requirement
Privacy	Role and organisation scoping on every query; public verification pages show validity only; confidential documents are audited on download.
Availability and recovery	Hosted on managed services. Database point-in-time recovery and daily backups are provided by Supabase. The platform records restore drills.
Performance	Interactive pages answer in a second or two for the expected scale (hundreds of trainees, dozens of programmes a year). Lists are paged. Not yet load-tested.
Usability	Plain language, empty states that say what to do next, confirmations before destructive actions, works on phones.
Accessibility	Labelled form fields, visible keyboard focus, sufficient colour contrast in both themes, reduced-motion support.
Maintainability	Typed code, versioned database migrations, an automated test suite of 40 end-to-end tests plus unit tests, written settings registry.
Auditability	Every significant change is written to the audit log with before and after values.

1.8 Success measures

Measure	Target
Time from programme end to released results	Within two working days
Certificates issued from evidence, not typed in	100%
Trainees registered through the platform instead of spreadsheets	100% of new programmes
Clients able to see their own training status without asking HAAB	Every active client
Outstanding invoices visible on one screen	Always current
Cross-client data leaks found in testing	Zero

1.9 Assumptions and constraints

- For now **client organisations carry the cost of their trainees**. Course fees are therefore hidden from the catalogue by default and Finance is hidden from trainees if HAAB chooses.
- The first release is invite-only and private. Public sign-up is a later switch, not a rebuild.
- Hosting is on Render (web, API, scheduled job) and Supabase (database, sign-in, files). Email is sent through Resend.
- The currency is Ghana cedi (GHS) by default; tax rate and due period are settings.
- Recognition details of a course (approving body, reference, expiry) are held in the database but hidden from screens until HAAB wants them.

1.10 Release status and roadmap

Phase	Content	Status
Phase 1: core	Accounts, catalogue, programmes, registration, attendance, assessments, results, certificates, finance, compliance, documents, reports, audit.	Complete on staging

Phase	Content	Status
Phase 2: corporate	Client portal, bulk registration, programmes by client, reservations, client dashboards.	Complete on staging
Hardening	Security audit and fixes, menu access, instructor materials, fee controls, reset tools.	Complete on staging
Before go-live	Penetration test, data-protection review, Supabase SMTP, custom domain, real course data, load test, backup and restore drill.	Planned
Phase 3: growth	Online payments, SMS and WhatsApp, SSO, hosted content, signature images, AI features.	Planned

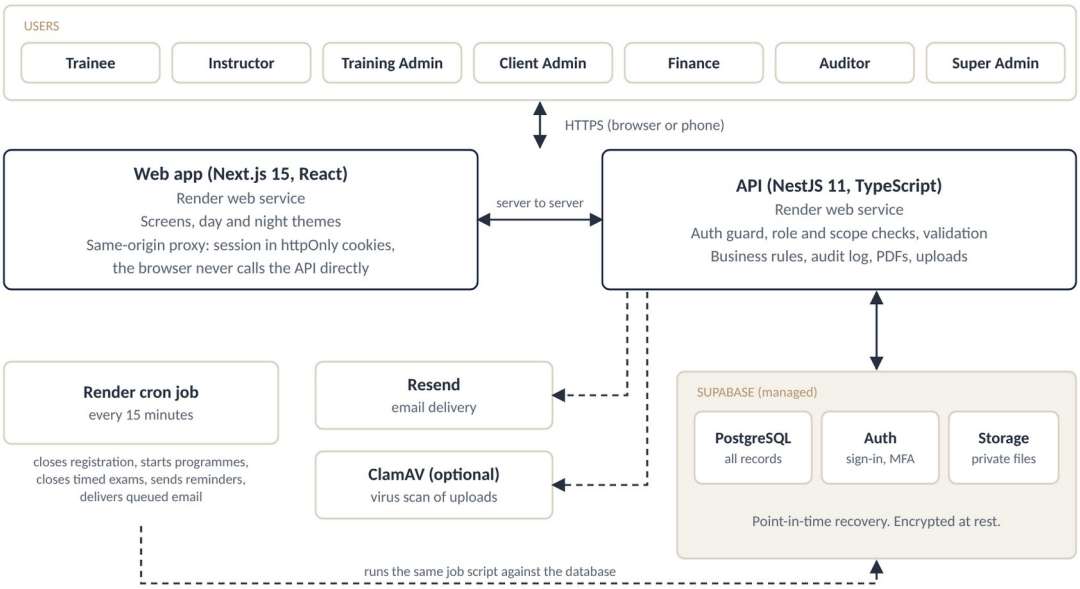
1.11 Known limits and risks

- Menu Access changes what each role sees. The real permissions are fixed in code and are not editable on screen.
- The content-security policy still allows inline styles and scripts needed by the front-end framework.
- Virus scanning is inactive until a ClamAV service is connected and configured.
- The audit chain is hashed but not keyed. Someone with full database access could rebuild it; Supabase access must be tightly held.
- Performance targets have not been load-tested.
- See docs/SECURITY-AUDIT.md in the repository for the accepted and open security items.

Part 2. Technical Requirements Document

2.1 Architecture

The platform is a three-part web system on managed hosting. The browser talks only to the **web app**. The web app talks to the **API** server to server. The API owns every rule and every query, and is the only part that talks to the database, file storage, the sign-in provider and the email provider.



System architecture

Component	Responsibility	Hosting
Web app	Screens, themes, navigation, forms. A same-origin proxy (`/api/...`) forwards calls to the API, adds the shared secret and the real client address, and keeps the session tokens in httpOnly cookies so page scripts never see them.	Render web service (Node 22)
API	Sign-in checks, authorisation, validation, all business rules, database access, PDFs, file handling, audit log, notifications.	Render web service (Node 22)
Scheduled job	Every 15 minutes: closes registration at the deadline, starts programmes, closes timed-out exams, sends certificate-expiry and class reminders, delivers queued email. Safe to run twice.	Render cron job
Database	All records. Row-level security is on with no policies, so Supabase's auto-generated REST interface exposes nothing; only the API connects.	Supabase PostgreSQL
Sign-in	Password verification, invitations, resets, two-step sign-in. The API verifies the tokens it issues.	Supabase Auth
Files	Uploaded documents and course material in a private bucket; the API streams them after checking permission.	Supabase Storage
Email	Transactional email through an HTTPS API; the key is held encrypted in the database and managed by a Super Administrator.	Resend

Component	Responsibility	Hosting
Virus scan	Optional. Uploads are streamed to ClamAV when a host is configured.	ClamAV service (optional)

2.2 Technology stack

Layer	Technology
Language	TypeScript 5.9 on Node.js 22 throughout
Web	Next.js 15 (App Router), React 19; plain CSS with design tokens, no UI framework; fonts Cormorant Garamond, Barlow and Barlow Condensed
API	NestJS 11 on Express; zod 4 for validation; `@nestjs/throttler` for rate limiting; helmet for headers
Database	PostgreSQL 16 (Supabase) accessed with raw SQL through `pg`; extensions btree_gist and pg_trgm
Documents	pdfkit for certificates, invoices and receipts; exceljs and CSV for exports and imports; JSZip for reading Word and Excel files
Security libraries	AES-256-GCM through Node crypto; zip-bomb guard for uploaded Office files; magic-byte upload inspection
Tests	Node test runner: 40 end-to-end tests against a real database, plus unit tests for dates, CSV, result rules and Word import
Delivery	npm workspaces monorepo (`apps/api`, `apps/web`, `db`, `scripts`); GitHub; Render blueprint files for production and staging

2.3 Repository layout

Path	Contents
`apps/api/src/modules`	One file per domain: users, organizations, courses, programmes, enrollments, attendance, assessments, results, certificates, finance, compliance, documents, dashboard, reports, settings, audit, deletions, workflow
`apps/api/src/common`	Guards, decorators, permissions, scope rules, database service, audit service, notification, mailer, storage, numbering, PDF generators, validation helpers, settings registry
`apps/api/src/scripts`	Command-line tools: jobs, seed, seed-catalogue, seed-sample, create-super-admin, reset-data
`apps/web/src/app`	Pages. `(app)` holds the signed-in screens; `login`, `register`, `set-password` and `verify` are public; `api/[...path]` is the proxy
`apps/web/src/components`, `lib`, `styles`	Shared components, API client and auth context, navigation rules, tokens
`db/migrations`	Ten ordered SQL migrations (0001 to 0010) applied by `scripts/migrate.mjs`

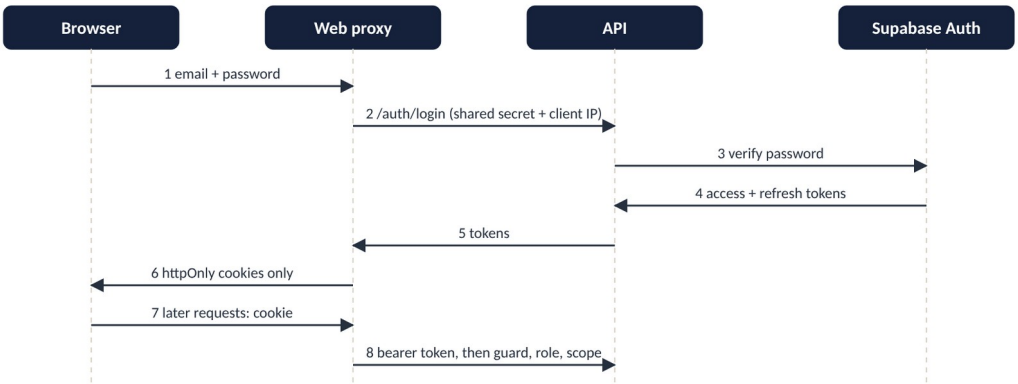
Path	Contents
`docs`	Deployment checklist, security audit, brand reference, UI structure, requirement traceability

2.4 Environments and deployment

- **Local:** `AUTH_MODE=dev` signs its own tokens with one shared password. The API refuses to start in this mode unless the database is on the same machine, so it can never run against a hosted database by accident.
- **Staging:** a separate Supabase project and Render services from `render.staging.yaml`. Sample data can be added with `seed-sample` (staging only).
- **Production:** `render.yaml`, deploying the `production` branch only. A pre-deploy step runs `scripts/migrate.mjs` before each release. Secrets are prompted once and never stored in git.
- **Release flow:** work on a feature branch, deploy to staging, test, then merge to `production`.
- **Health:** `GET /health` on the API and `/login` on the web app are the health checks.

2.5 Authentication and sessions

The API supports two modes. In **supabase** mode (all hosted environments) the password is checked by Supabase Auth and the API verifies the tokens it receives. In **dev** mode the API signs its own tokens for local work. The browser never receives a token.



Sign-in sequence

- Access and refresh tokens are stored in **httpOnly, SameSite=Lax, Secure** cookies set by the web app. The proxy refreshes the access token when it expires.
- The proxy forwards only a fixed list of path segments, rejects dot segments and encoded separators, and always blocks direct calls to the sign-in routes from other origins.
- The real client address is trusted only when a request carries the shared proxy secret. It drives rate limiting and the audit log.
- **Two-step sign-in** is optional. Anyone can enrol an authenticator app; a Super Administrator can require it for chosen roles, in which case the session stays restricted until the code is verified.
- Invitations and password resets use Supabase links that return to `/set-password`.

2.6 Authorisation

Every route is protected by a global guard. A route declares the permissions it needs with `Require(...)`, and any one of the listed permissions is enough. Beyond the role, **scope rules** limit what each person can reach: a Client Administrator sees only their own organisation, a trainee only themselves, an instructor only programmes they lead or teach. Records outside someone's scope answer **404**, never 403, so their existence is not revealed.

Permission matrix

Permission	Super	Training	Instr.	Trainee	Client	Finance	Auditor
`users:read`	Yes	Yes			Yes		Yes
`users:write`	Yes	Yes			Yes		
`orgs:read`	Yes	Yes			Yes	Yes	Yes
`orgs:write`	Yes	Yes					
`courses:read`	Yes	Yes	Yes	Yes	Yes	Yes	Yes
`courses:write`	Yes	Yes					
`programmes:read`	Yes	Yes	Yes	Yes	Yes	Yes	Yes
`programmes:write`	Yes	Yes					
`enrollments:read`	Yes	Yes	Yes	Yes	Yes	Yes	Yes
`enrollments:write`	Yes	Yes			Yes		
`trainees:register`	Yes	Yes	Yes				
`materials:write`	Yes	Yes	Yes				
`attendance:read`	Yes	Yes	Yes				Yes
`attendance:write`	Yes	Yes	Yes				
`questions:write`	Yes	Yes	Yes				
`assessments:read`	Yes	Yes	Yes	Yes			Yes
`assessments:write`	Yes	Yes	Yes				
`attempts:mark`	Yes	Yes	Yes				
`results:read`	Yes	Yes	Yes	Yes	Yes		Yes
`results:finalise`	Yes	Yes					
`results:override`	Yes	Yes					
`certificates:read`	Yes	Yes		Yes	Yes		Yes
`certificates:issue`	Yes	Yes					
`certificates:revoke`	Yes	Yes					
`invoices:read`	Yes	Yes		Yes	Yes	Yes	Yes
`invoices:write`	Yes					Yes	
`payments:write`	Yes					Yes	
`documents:read`	Yes	Yes	Yes	Yes	Yes	Yes	Yes
`documents:write`	Yes	Yes	Yes	Yes	Yes	Yes	
`documents:delete`	Yes	Yes					
`compliance:read`	Yes	Yes			Yes		Yes
`compliance:write`	Yes	Yes					
`reports:read`	Yes	Yes	Yes		Yes	Yes	Yes
`audit:read`	Yes						Yes
`settings:read`	Yes	Yes					Yes

Permission	Super	Training	Instr.	Trainee	Client	Finance	Auditor
`settings:write`	Yes						
`integrations:manage`	Yes						

- `results:finalise` can also be granted to one instructor by a Super Administrator (`users.can_finalise_results`). It then applies only to programmes that instructor leads or teaches.
- Menu Access (a Super Administrator setting) hides or shows menu items per role. It never grants a permission.
- A global **fee-hiding interceptor** removes price fields from every response sent to an instructor, so a fee cannot leak through any screen or export.
- Segregation of duties is enforced in the API: a marker cannot finalise what they marked; finance and training permissions do not overlap.

2.7 API design

- **Style:** JSON over HTTPS, resource-oriented paths, standard verbs. Lists accept `limit`, `offset` and `q`, and return `{ data, total }`.
- **Validation:** every body and query is parsed with zod; failures answer **400** with a list of issues. Updates use a `patchOf` helper so an omitted field is never reset to its default.
- **Errors:** `401` not signed in, `403` not allowed, `404` not found or out of scope, `409` a rule stops it (message explains what to do instead), `429` throttled.
- **Limits:** 300 requests a minute by default per client, with tighter limits on sign-in, uploads, exams, public verification and imports.
- **Transactions:** multi-step changes run in one database transaction with row locks where order matters.
- **Public routes:** only sign-in, set-password, forgot-password, health, certificate verification and the registration request form.

Endpoint catalogue

All routes of the API. The permission column shows what the route requires. "Signed in" means any authenticated user, with the rule enforced inside the handler (for example by settings page or scope).

Assessments, questions and exams

Method	Path	Permission
GET	`/questions`	questions:write assessments:read
POST	`/questions`	questions:write
POST	`/questions/import`	questions:write
PATCH	`/questions/:id`	questions:write
GET	`/assessments`	assessments:read
GET	`/assessments/:id`	assessments:read
POST	`/assessments`	assessments:write
PATCH	`/assessments/:id`	assessments:write
PUT	`/assessments/:id/questions`	assessments:write

Method	Path	Permission
POST	`/assessments/:id/questions/auto`	assessments:write
POST	`/assessments/:id/publish`	assessments:write
POST	`/assessments/:id/close`	assessments:write
POST	`/assessments/:id/release`	attempts:mark
GET	`/my/assessments`	assessments:read
POST	`/assessments/:id/start`	assessments:read
GET	`/attempts/:id`	assessments:read
PUT	`/attempts/:id/answers`	assessments:read
POST	`/attempts/:id/submit`	assessments:read
GET	`/attempts`	attempts:mark
GET	`/attempts/:id/review`	attempts:mark
PUT	`/attempts/:id/mark`	attempts:mark

Users, trainees, registration and imports

Method	Path	Permission
GET	`/users`	users:read
GET	`/users/by-organization`	users:read
GET	`/users/:id`	users:read
GET	`/users/:id/profile`	users:read
PUT	`/users/:id/profile`	users:write
GET	`/profile`	Signed in
PUT	`/profile`	Signed in
POST	`/users`	users:write
GET	`/registration/status`	public
POST	`/registration`	public
GET	`/registrations`	users:write
POST	`/registrations/:id/approve`	users:write
POST	`/registrations/:id/reject`	users:write
GET	`/trainees/register-options`	trainees:register
POST	`/trainees/import-file`	trainees:register
POST	`/users/import`	users:write
PATCH	`/users/:id`	users:write
POST	`/users/:id/suspend`	users:write
POST	`/users/:id/send-reset`	users:write
POST	`/users/:id/reset-mfa`	users:write
POST	`/users/:id/reactivate`	users:write

Courses, categories, modules and materials

Method	Path	Permission
GET	`/course-categories`	courses:read
POST	`/course-categories`	courses:write

Method	Path	Permission
PATCH	`/course-categories/:id`	courses:write
DELETE	`/courses/:id`	courses:write
DELETE	`/course-categories/:id`	courses:write
GET	`/courses`	courses:read
GET	`/courses/:id`	courses:read
POST	`/courses`	courses:write
PATCH	`/courses/:id`	courses:write
PUT	`/courses/:id/prerequisites`	courses:write
PUT	`/courses/:id/instructors`	courses:write
POST	`/courses/:id/modules`	courses:write materials:write
DELETE	`/modules/:id`	courses:write materials:write
POST	`/modules/:id/materials`	courses:write materials:write
POST	`/modules/:id/materials/upload`	courses:write materials:write
GET	`/materials/:id/file`	courses:read
DELETE	`/materials/:id`	courses:write materials:write
PUT	`/materials/:id/progress`	courses:read
GET	`/my/learning/:courseId`	courses:read
GET	`/my/learning`	courses:read

Programmes and sessions

Method	Path	Permission
GET	`/classrooms`	programmes:read
POST	`/classrooms`	programmes:write
PATCH	`/classrooms/:id`	programmes:write
GET	`/programmes`	programmes:read
GET	`/programmes/by-client`	programmes:read
GET	`/programmes/:id`	programmes:read
POST	`/programmes`	programmes:write
PATCH	`/programmes/:id`	programmes:write
POST	`/programmes/:id/status`	programmes:write
GET	`/programmes/:id/sessions`	programmes:read
POST	`/programmes/:id/sessions`	programmes:write
PATCH	`/sessions/:id`	programmes:write
DELETE	`/sessions/:id`	programmes:write
GET	`/schedule`	programmes:read

Settings, menu and integrations

Method	Path	Permission
GET	`/settings/pages`	Signed in
GET	`/settings/menu`	Signed in
PUT	`/settings/menu`	Signed in
GET	`/settings/roles`	Signed in
GET	`/settings/pages/:key`	Signed in
PUT	`/settings/pages/:key`	Signed in
GET	`/settings/system-status`	integrations:manage
GET	`/integrations`	integrations:manage
PUT	`/integrations/:key`	integrations:manage
POST	`/integrations/:key/test`	integrations:manage

Authentication

Method	Path	Permission
POST	`/auth/login`	public
POST	`/auth/refresh`	public
POST	`/auth/forgot`	public
POST	`/auth/set-password`	public
GET	`/auth/me`	Signed in
POST	`/auth/mfa/enroll`	Signed in
POST	`/auth/mfa/verify`	Signed in
POST	`/auth/mfa/disable`	Signed in
POST	`/auth/logout`	Signed in

Certificates

Method	Path	Permission
GET	`/verify/:token`	public
GET	`/certificates`	certificates:read
GET	`/certificates/:id`	certificates:read
GET	`/certificates/:id/pdf`	certificates:read
POST	`/certificates/:id/revoke`	certificates:revoke
POST	`/certificates/:id/reissue`	certificates:revoke certificates:issue
GET	`/certificate-templates`	certificates:read
POST	`/certificate-templates`	certificates:issue
PATCH	`/certificate-templates/:id`	certificates:issue

Super Administrator deletions

Method	Path	Permission
DELETE	`/users/:id`	settings:write

Method	Path	Permission
DELETE	`/organizations/:id`	settings:write
DELETE	`/programmes/:id`	settings:write
DELETE	`/enrollments/:id`	settings:write
DELETE	`/classrooms/:id`	settings:write
DELETE	`/assessments/:id`	settings:write
DELETE	`/questions/:id`	settings:write
DELETE	`/certificate-templates/:id`	settings:write

Results

Method	Path	Permission
GET	`/results`	results:read
POST	`/programmes/:id/results/compute`	results:finalise attempts:mark
POST	`/results/:id/finalise`	results:finalise
POST	`/programmes/:id/results/finalise-all`	results:finalise
POST	`/results/:id/override`	results:override
POST	`/programmes/:id/results/release`	results:finalise
GET	`/results/:id/changes`	results:read

Dashboard, search, notifications and health

Method	Path	Permission
GET	`/dashboard`	Signed in
GET	`/search`	Signed in
GET	`/notifications/mine`	Signed in
POST	`/notifications/read-all`	Signed in
POST	`/notifications/:id/read`	Signed in
GET	`/health`	public

Finance

Method	Path	Permission
GET	`/invoices`	invoices:read
GET	`/invoices/:id`	invoices:read
POST	`/invoices`	invoices:write
POST	`/invoices/:id/cancel`	invoices:write
POST	`/invoices/:id/payments`	payments:write
POST	`/payments/:id/refund`	payments:write
GET	`/invoices/:id/pdf`	invoices:read
GET	`/payments/:id/receipt`	invoices:read

Attendance

Method	Path	Permission
GET	`/sessions/:id/attendance`	attendance:read
PUT	`/sessions/:id/attendance`	attendance:write
GET	`/sessions/:id/qr`	attendance:write
POST	`/attendance/qr-checkin`	Signed in
GET	`/programmes/:id/attendance-summary`	attendance:read

Compliance

Method	Path	Permission
GET	`/compliance/overview`	compliance:read
GET	`/compliance/gaps`	compliance:read
GET	`/compliance/requirements`	compliance:read
POST	`/compliance/requirements`	compliance:write
DELETE	`/compliance/requirements/:id`	compliance:write

Registrations

Method	Path	Permission
GET	`/enrollments`	enrollments:read
POST	`/enrollments`	enrollments:write courses:read
POST	`/programmes/:id/enrol`	enrollments:write trainees:register
POST	`/enrollments/:id/confirm`	enrollments:write
POST	`/enrollments/:id/cancel`	enrollments:read

Clients (organisations)

Method	Path	Permission
GET	`/organizations`	orgs:read
GET	`/organizations/:id`	orgs:read
GET	`/organizations/:id/summary`	orgs:read
POST	`/organizations`	orgs:write
PATCH	`/organizations/:id`	orgs:write

Documents

Method	Path	Permission
GET	`/documents`	documents:read
POST	`/documents`	documents:write
GET	`/documents/:id/download`	documents:read
DELETE	`/documents/:id`	documents:delete

Audit log

Method	Path	Permission
GET	`/audit`	audit:read
GET	`/audit/export`	audit:read
GET	`/audit/verify`	audit:read

Reports

Method	Path	Permission
GET	`/reports`	reports:read
GET	`/reports/:key`	reports:read

To-do lists and close-out

Method	Path	Permission
GET	`/dashboard/todo`	Signed in
GET	`/programmes/:id/closeout`	results:read

2.8 Data access and migrations

- **Raw SQL** with parameters, through a small database service that offers `query`, `one` and `tx` (transaction). There is no ORM. Parameters are always bound, never concatenated.
- **Constraints in the database** back the rules in the code: unique keys, check constraints for statuses, foreign keys, and exclusion constraints (btree_gist) that refuse double booking of a classroom, an instructor or a programme.
- **Migrations** are numbered SQL files applied in order by `scripts/migrate.mjs`, each recorded in `schema_migrations`. They run before each release.
- **Row-level security** is enabled on every table with no policies. Only the API role can read or write.
- **Triggers:** `updated_at` on main tables, and the audit-log chain and immutability triggers.

2.9 Security controls

Threat	Control
Broken access control	Server-side permission check on every route; scope filters in every query; 404 for out-of-scope records; menu hiding is cosmetic only.
Injection	Parameterised SQL; zod validation of all input; no raw HTML is rendered.
Cross-site scripting and clickjacking	Content-Security-Policy, X-Frame-Options DENY, no-sniff and referrer headers; links on learning material restricted to http and https.
Cross-site request forgery	SameSite cookies and same-origin checks in the proxy.
Credential attacks	Per-address and per-account throttling; generic answers that never reveal whether an account exists; optional two-step sign-in.
Session theft	httpOnly Secure cookies; tokens never reach page scripts; refresh handled by the proxy.

Threat	Control
Malicious uploads	Type decided by file content (magic bytes), size limit, zip-bomb guard for Office files, optional ClamAV scan, private storage, downloads sent as attachments.
Data exposure	ID numbers and integration keys encrypted at rest (AES-256-GCM); write-only secrets; masked values on screen; fee-hiding for instructors.
Tampering	Append-only, hash-chained audit log with a verification function; certificates carry unguessable tokens; finalised results and issued certificates cannot be changed in place.
Abuse of privileged roles	Maker-checker on results; reasons required for overrides and revocations; Super Administrator-only deletions; official record cannot be deleted.
Misconfiguration	Dev sign-in refused unless the database is local; Supabase mode requires its keys; secrets only in environment variables.

2.10 Background jobs

A single script (`jobs`) runs every 15 minutes. Each step is idempotent, and queued email is locked with `skip locked` so two overlapping runs never send the same message twice.

Step	Effect
Close registration	Programmes past their registration deadline move to Registration closed.
Start programmes	Programmes whose start date has arrived move to Ongoing.
Close attempts	Timed exam attempts past their time limit are submitted and marked where possible.
Expiry reminders	Certificate-expiry reminders at the configured number of days before expiry.
Session reminders	Class reminders before each session.
Deliver email	Sends queued emails; up to five attempts each; failures are recorded.

2.11 Files and storage

- Files go to a private Supabase Storage bucket (or a local folder in development). Keys are generated by the platform; names given by users are never used as paths.
- An upload is inspected by content, not by its extension. Allowed types and the largest size are settings (File Types). Office files are scanned for zip bombs.
- Every download passes through the API, which checks the person may see that file. Downloads of confidential files are written to the audit log.
- Uploaded course material creates a document of category `course_material`; a trainee confirmed on the course, the course's instructors and staff may download it.

2.12 Notifications and email

A notification creates an in-app notice and, when email is enabled, a queued email. Emails to a registered person end with `Sign in: <platform address>/login`. Sender name, reply-to address and signature are settings; the provider key is held encrypted. Reminders carry a de-duplication key so they are sent once.

2.13 Generated documents

Certificates, invoices and receipts are PDFs produced by the API with pdfkit, using the HAAB logo and embedded fonts. The certificate shows the issuer name, course banner, number, date and a verification QR code; the footer carries only the company name. Reports export to PDF, Excel and CSV.

2.14 Audit log

Every significant action is written inside the same transaction as the change, with the actor, role, address, record and before and after values. A database trigger links each row to the previous one by a SHA-256 hash and another trigger blocks any update, delete or truncate. `audit_verify()` re-computes the chain and reports the first broken link. Super Administrators and Auditors can read the log and run the verification.

2.15 Configuration

Variable	Service	Purpose
`DATABASE_URL`, `DATABASE_SSL`, `DATABASE_POOL_MAX`	API	Database connection (use the Supabase transaction pooler in production)
`AUTH_MODE`	API	`supabase` for hosted use; `dev` only with a local database
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_JWT_SECRET`	API	Sign-in provider and storage
`DATA_ENCRYPTION_KEY`	API	32 random bytes, base64. Encrypts ID numbers and integration keys. Losing it makes them unreadable, so keep a sealed copy.
`STORAGE_DRIVER`, `SUPABASE_STORAGE_BUCKET`, `STORAGE_DIR`, `MAX_UPLOAD_MB`	API	Where files go and the largest upload
`CLAMAV_HOST`, `CLAMAV_PORT`	API	Optional virus scanning
`RESEND_API_KEY`, `MAIL_FROM`	API	Email (the key can also be set on screen under API & Integrations)
`WEB_ORIGIN`, `PUBLIC_WEB_URL`	API	Allowed browser origin and the address used in emails and certificate QR links
`PROXY_SHARED_SECRET`	API and web	Shared secret (at least 24 characters) between the proxy and the API. Must be identical.
`API_URL`, `TRUST_CLOUDFLARE`	Web	Where the API is, and whether to trust Cloudflare's client-address header
`LOGO_PATH`	API	Logo printed on documents (defaults to the one shipped with the web app)
`NODE_ENV`, `PORT`	Both	Standard

2.16 Settings registry

Administrative settings are defined once in a registry (key, title, schema, defaults, form fields, who may write). The Settings screens are generated from it, and every change is validated and audited. Secrets are never returned.

Key	Title	Purpose	Written by
`organization`	General Setting	HAAB legal name, registration, address, contact	Super Admin
`security`	Security Setting	Roles that must use two-step sign-in	Super Admin
`registration`	Trainee Registration	Allow account requests from the sign-in page	Super Admin
`public_access`	Public Access	Open the platform to the general public	Super Admin
`trainee_access`	Trainee Access	Whether trainees see Finance	Super Admin
`course_fees`	Course Fees	Whether course fees are shown in the catalogue	Super Admin
`menu_access`	Menu Access	Menu items shown or hidden per role	Super Admin
`period`	Training Period	Month the reporting year starts	Super Admin
`training`	Training Rules	Default pass mark and attendance, expiry window, marker-cannot-finalise	Super Admin
`notifications`, `email`	Notification and Email Setting	Which events send email, sender, reply-to, signature	Super Admin
`payment_methods`, `finance`, `print`	Payment, Currency, Print	Methods, tax, due days, currency, print header and footer	Super Admin, Finance (and Training for print)
`modules`, `file_types`, `backup`	Modules, File Types, Backup	Feature switches, upload rules, recovery targets and drill log	Super Admin
`whatsapp`, `sms`, `languages`, `captcha`	Planned	Reserved for later releases	Super Admin

2.17 Testing and quality

- **40 end-to-end tests** start the real API against a real PostgreSQL database and cover access control, client isolation, the full training flow, finance, uploads, settings, security hardening, deletions, reports and every feature in this document.
- Unit tests cover dates, CSV, result rules and Word import.
- The web app is type-checked and production-built in the same pipeline; screens were also checked in a real browser at desktop and phone widths.
- Continuous integration runs build and tests on every push.

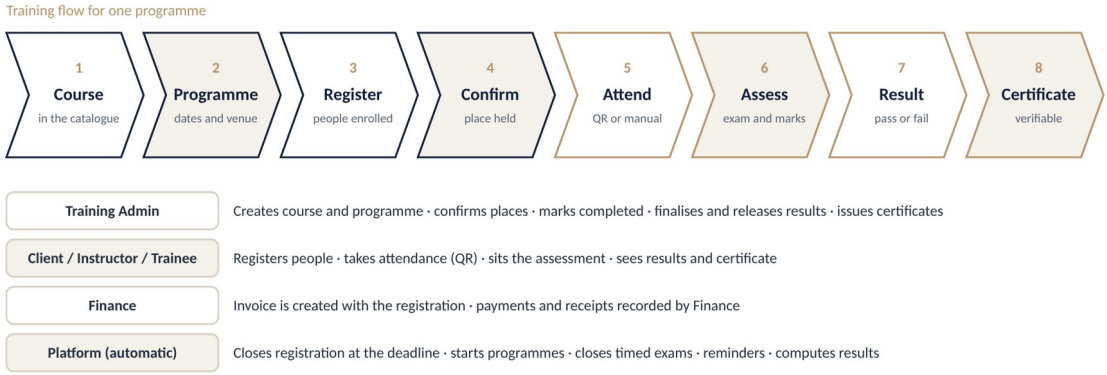
2.18 Operations

- **Backups:** Supabase provides backups and, on supported plans, point-in-time recovery. Keep Auto-Deploy off for production and record restore drills in the Backup Restore setting.
- **Monitoring:** health endpoints, host logs, and failed-email and audit views on screen.
- **Key custody:** `DATA_ENCRYPTION_KEY` and `PROXY_SHARED_SECRET` must be backed up outside the host.
- **Clearing data:** `npm run reset-data -w apps/api` previews and, with `RESET_DATA=yes`, removes everything except Super Administrator accounts (settings, templates, catalogue and audit log are kept unless told otherwise).

Part 3. Application Flow

This part follows the work end to end. Each flow lists who acts, what the platform does and the rule that applies at each step. Status tables at the end define every legal change of state.

3.1 The whole training cycle



One programme from catalogue to certificate

3.2 Getting in

A. Invitation (the normal way)

1. A Super Administrator or Training Administrator (or a Client Administrator for their own trainees) creates the person with name, email and role. For a client user the organisation is chosen too.
2. The platform creates the sign-in account, marks the user **Invited**, writes an audit entry and sends the invitation email.
3. The person opens the link, chooses a password of at least 12 characters and sets it. Their status becomes **Active** on first sign-in.
4. They sign in. If their role requires two-step sign-in, they enrol an authenticator app and verify a code before anything else opens.
5. They land on the home page for their role, with a "To do" list.

B. Request an account (optional)

1. A Super Administrator switches on **Trainee Registration**. The sign-in page then offers **Request an account**.
2. The person sends name, email, phone and employer. The platform stores a request (status **Pending**); nothing else is created and nothing reveals whether the email already exists.
3. A HAAB administrator opens **Trainees → Registration requests**, then **Approves** (the account is created and the invitation sent, as in A) or **Rejects** with a reason.

C. Forgotten password and sign-out

The person asks for a link, receives it if the account exists (the page always answers the same), and sets a new password. Sessions end after a period of inactivity and refresh silently while the person is working. Repeated wrong passwords are throttled.

3.3 Where each role goes

Role	Home page shows	Main menu	Phone bottom bar
Trainee	My training, upcoming classes, assessments to take, results, certificates, To do	My training, Calendar, Check in, Assessments, Results, Certificates, Finance (if shown), Documents	Home, Training, Check in, Results
Instructor	My programmes and sessions, marking queue, To do	Register trainees, Courses, Programmes, Enrolments, Attendance, Assessments, Results, Documents, Reports	Home, Programmes, Attendance, Marking
Training Administrator	Training KPIs, charts, To do (registrations to confirm, results to finalise, certificates expiring)	Trainees, Clients, Instructors, Courses, Programmes, Enrolments, Attendance, Assessments, Results, Certificates, Compliance, Documents, Reports, Settings (read)	Home, Trainees, Programmes, Attendance
Super Administrator	As Training Administrator, plus platform health	Everything above plus Audit log and all System setting pages	Home, Trainees, Programmes, Attendance
Client Administrator	Their organisation: trainees, registrations, results, invoices, compliance	Trainees, Clients (own), Programmes, Enrolments, Results, Certificates, Finance, Compliance, Documents, Reports	Home, Trainees, Enrolments, Results
Finance Officer	Revenue, outstanding, recent payments	Clients, Programmes, Enrolments, Finance, Documents, Reports, Settings (payment pages)	Home, Invoices, Clients, Enrolments
Auditor	Compliance and audit summaries	Everything read-only, Audit log, Compliance, Reports	Home, Audit, Compliance, Reports

A Super Administrator can hide or show menu items per role under **System setting → Menu Access**. Hidden pages are also refused when typed into the address bar, but the server permissions are what actually protect the data.

3.4 Registering people on a programme

A. Corporate path (today)

1. A Training Administrator creates the programme and opens it for registration. It may be **reserved** for one client.
2. A Client Administrator (own trainees only), an Instructor (programmes they lead or teach) or a Training Administrator registers trainees, one by one or from an Excel list. The platform previews the list, reports every problem first, then creates accounts and invitations for new people and recognises people it already knows.

3. Each registration starts **Pending**. If the programme is full the person is **Waitlisted**; when a place opens the first person on the waiting list becomes Pending automatically.
4. An invoice is raised to the sponsoring organisation (or the trainee, when none) in the same transaction.
5. A Training Administrator **confirms** the place. If **Payment before confirmation** is on, an invoice that is not paid blocks it unless an override reason is recorded.
6. The trainee is told by email and in the app; they can now see the schedule and learning material.

B. Public path (later, when switched on)

1. A Super Administrator turns on **Public Access**.
2. Anyone can request an account; HAAB approves each request. The person has no organisation.
3. The trainee browses open programmes and registers themselves. An invoice is raised to them personally and Finance becomes visible to them.
4. They pay (recorded by Finance, or before confirmation if that setting is on); the place is confirmed.

3.5 Programme life cycle

From	To	Who	Rule
Draft	Open for registration	Training Admin	The course must be Active.
Open for registration	Registration closed	Training Admin or automatic at the deadline	
Registration closed	Open for registration	Training Admin	Reopen if places are needed.
Open or Closed	Ongoing	Training Admin or automatic on the start date	
Ongoing	Completed	Training Admin	Completes confirmed registrations and settles results even if later session dates were not reached.
Any before Completed	Cancelled	Training Admin	A reason is required. Registrations and unpaid invoices are cancelled.
Completed or Cancelled	(none)		Final. Cannot be edited.

3.6 Attendance

1. A Training Administrator or Instructor adds **sessions** to the programme: date, time, room, instructor. The platform refuses a clash of instructor, room or programme.
2. On the day the instructor opens the session roster. Either they mark each trainee **Present, Late, Absent or Excused** (method **manual**), or they show the **QR code** panel on the classroom screen.

3. The QR code changes every 30 seconds and carries a signed, time-limited code. A trainee opens **Check in**, scans it with their own phone, and is recorded as present (method **qr**). A photo of the code cannot be used later.
4. A QR scan never overwrites a mark the instructor made by hand. Attendance percentage is computed from the sessions actually held.

3.7 Assessments and marking

1. A Training Administrator or Instructor builds an assessment (exam, quiz or practical) for a course or programme, picks questions from the bank, sets duration, pass mark, attempts, open and close times and release mode. It is **Draft** until published.
2. On **Publish** the assessment opens for confirmed trainees within its time window. A trainee **starts** an attempt (status **In progress**). The server keeps the clock; answers auto-save as the trainee goes; the screen shows a countdown.
3. On **Submit** or time-out the attempt becomes **Submitted**. Objective questions (single or multiple choice, true or false, matching) and short answers (compared with the accepted answers) are marked automatically.
4. Essay and scenario answers go to the **Marking queue**. A marker gives marks and feedback; the attempt becomes **Marked**. Marks are sealed once the result is finalised.
5. The assessment can be **Closed** when it is published and no longer needed.

3.8 Results, release and close-out

1. The platform computes each trainee's result from evidence: attendance against the course minimum and the best attempt on every published assessment. Until the programme has ended or is Completed the result is **Pending**.
2. When the programme is **Completed** results settle to **Pass** or **Fail**. A person other than the marker **finalises** each result (maker-checker). A Super Administrator may let one instructor do this on their own programmes.
3. Overriding a computed result requires a written reason and is audited.
4. **Release results** makes them visible to the trainee and the client; a notification and email go out. Until release, trainees see nothing.
5. The **close-out checklist** on the programme page shows the single next step: start, complete, mark, recalculate, finalise, release, done.

3.9 Certificates

1. When a result is finalised as **Pass**, the platform issues a certificate once (never twice for one registration): number ATMP-year-CATEGORY-serial, expiry from the course validity, template and QR code.
2. The trainee and the client see it and can download the PDF. Staff can **reissue** (keeps the original dates), **renew** (a new certificate linked to the old one) or **revoke** with a reason (with an optional notice to the holder).
3. Anyone can open the QR link or enter the verification address. The public page shows only whether it is **Valid, Expiring soon, Expired or Revoked**, with name, course and dates, and nothing else.
4. The scheduled job sends expiry reminders at the configured number of days before expiry.

3.10 Finance

1. An invoice is raised automatically for a registration that carries a fee, using the tax rate and due period in Currency settings, or manually by Finance. Invoice status: **Pending**.
2. Finance records a **payment** (method, amount, reference). A partial payment makes the invoice **Partially paid**; settling it makes it **Paid** and issues a receipt.
3. A refund is recorded with a reason; failed or cancelled payments are kept on the record. Cancelling a programme cancels unpaid invoices.
4. Clients see their own invoices and receipts; trainees see theirs if Finance is switched on for them. Instructors never see fees.
5. Recorded payments are the official record and are never deleted.

3.11 Learning material

1. A Training Administrator or an instructor who teaches the course opens **Courses → course → Modules and materials**.
2. They **Add module**, then **Add material** as an uploaded file (PDF, Word, PowerPoint, Excel, image or text; checked by content) or a web link (http or https only). Material can be marked required.
3. Trainees confirmed on a programme of the course see the material, download uploaded files and mark progress (Not started, In progress, Completed). Other trainees do not see it.
4. An instructor can remove only what they added; staff can remove anything.

3.12 Compliance, documents and reports

- **Compliance:** staff define requirements (a course every N months for a client or aviation role, with a grace period). The overview shows who is up to date, expiring, expired or missing, and attendance shortfalls.
- **Documents:** staff, instructors, clients and trainees upload documents by category; access follows role and confidentiality; downloads of confidential files are audited.
- **Reports:** nine reports with filters and export to PDF, Excel and CSV. Exports are audited. Clients get their own organisation only.
- **Search:** the top-bar search finds trainees, courses, programmes, certificates, organisations and instructors, limited to what the role may see.

3.13 Settings, deletions and reset

- **Settings** are changed on System setting pages; each change is validated and written to the audit log. Switches that change what people see (Course Fees, Trainee Access, Public Access, Menu Access) take effect for the Super Administrator at once and for others the next time they open the platform.
- **Deleting:** only a Super Administrator sees Delete buttons. The confirmation says what goes with the record. If a certificate, finalised result or recorded payment depends on it, the delete is refused and nothing changes.
- **Reset:** a command-line tool previews and then, with explicit confirmation, empties the platform except Super Administrator accounts.

3.14 State definitions

Object	States	Notes
User	Invited, Active, Suspended	Suspending keeps history and blocks sign-in.
Programme	Draft, Open for registration, Registration closed, Ongoing, Completed, Cancelled	See 3.5.
Registration	Pending, Confirmed, Waitlisted, Completed, Cancelled, No-show	Waitlist promotes automatically.
Attendance	Present, Absent, Late, Excused	Method: manual or QR.
Assessment	Draft, Published, Closed	Only a published one can be taken or closed.
Attempt	In progress, Submitted, Marked	Server clock decides time-out.
Result	Pending, Pass, Fail, Absent, Disqualified	A finalised result is never recalculated.
Certificate	Valid, Revoked	Expiry is a date; "Expiring soon" and "Expired" are derived.
Invoice	Pending, Partially paid, Paid, Failed, Refunded, Cancelled	
Payment	Paid, Failed, Refunded, Cancelled	Never deleted.
Registration request	Pending, Approved, Rejected	
Course	Draft, Active, Archived	Only Active courses can open programmes.
Question	Active, Retired	A question used in an exam can only be retired, not deleted.

Part 4. UI/UX Design Brief

4.1 Design intent

The platform is a working tool for people who spend hours in it: training administrators, instructors and finance staff. It must also be easy for a trainee opening it once on a phone in a classroom. The design therefore pairs **HAAB's premium aviation brand** (the dark navy-black and gold of haabaviation.com) with a **calm, warm work area** that keeps dense tables readable.

Principle	What it means in practice
Show only what is yours	Every role sees its own menu, its own numbers and its own to-do list. Things a role cannot use are not drawn. Nothing about other clients is ever hinted at.
Say the next step	Every empty list explains what to do next. Programmes show a close-out checklist. Home pages open with a To do list with counts.
Plain language	Short sentences, no jargon, no internal abbreviations (the label says "registration", not "enrollment record").
Confirm what cannot be undone	Deleting, cancelling, revoking and finalising always ask first and say what will happen. A refused action explains why and what to do instead.
One system, two themes	Day (warm paper) and Night (the website's own dark) are built from the same tokens, so both always agree.
Phone first for trainees	Check-in, assessments, results and certificates work comfortably with one thumb.

4.2 Brand

Colour

Token	Day	Night	Use
`--bg`	#F7F5F1	#0A0C0F	Page background
`--surface`	#FFFFFF	#111418	Cards, tables, forms
`--surface-2`	#F1EEE8	#181D24	Table headers, raised areas
`--text` / `--text-muted`	#14181D / #5A6470	#E8E4DC / #9EA8B4	Body and secondary text
`--accent`	#B8966E (gold)	#B8966E	Primary buttons, active bars, fills
`--accent-text`	#86683F	#D4B896	Gold used as text (day uses the darker gold to pass contrast)
`--chrome-bg`	#0A0C0F	#06080A	Sidebar and top bar, dark in both themes like the website
`--ok` `--warn` `--danger` `--info`	#2F6B4F #8A5A0B #9B2C2C #2C5282	Tints of the same hues	Status badges and alerts, each with a soft background

Components use **semantic tokens only**, never raw colours. The theme is chosen by the device by default and can be switched from the top bar; the choice is remembered and applied before first paint, so there is no flash. Printed output (certificates, invoices, reports) is always on white.

Typography

Role	Face	Spec
Display and page titles	Cormorant Garamond	Light 300 at 38px for the page title with an italic gold accent word ("Course *catalogue*"); 26px and 20px for section titles; lining figures for all numbers
Labels, navigation, buttons, table headers	Barlow Condensed	Uppercase, 600 weight, 12 to 13px, wide tracking (0.16em; buttons 2.5px)
Body and forms	Barlow	15px, 1.5 line height, 400 weight

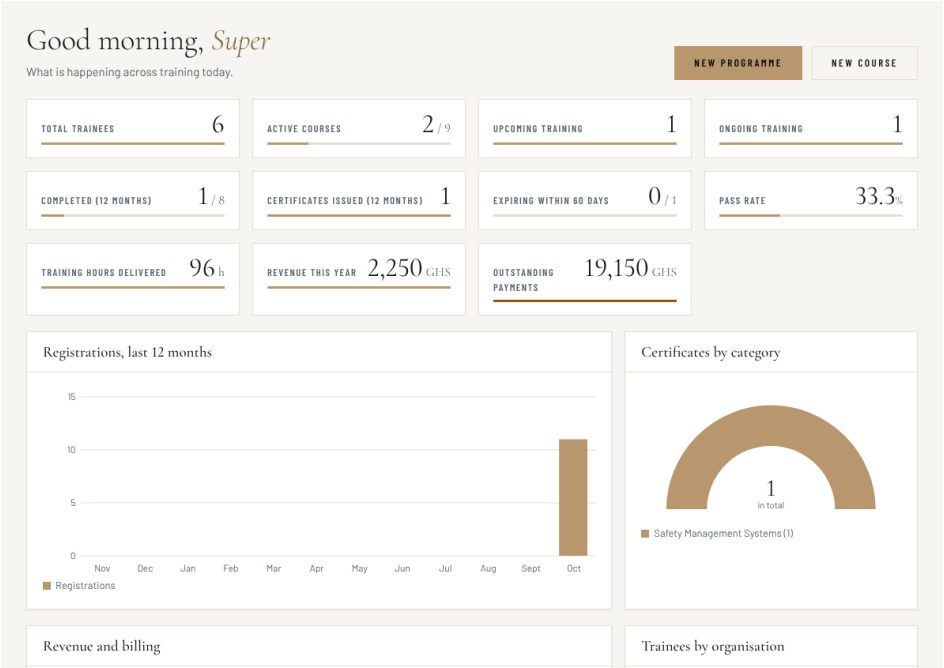
Shape and tone

- Square corners everywhere (radius 0). One-pixel outlined tags. Faint shadows only.
- Primary button: solid gold with dark text. Secondary: outlined. Destructive: red outline. Link buttons: gold text.
- Voice: plain, authoritative, outcome-focused, in British English. Legal name on documents: HAAB Aviation Consultancy Services Ltd.
- The logo file is used as supplied: white on the dark sidebar and login, colour on certificates, invoices and light backgrounds. It is never redrawn or recoloured by hand.

4.3 Layout and navigation

Desktop and tablet

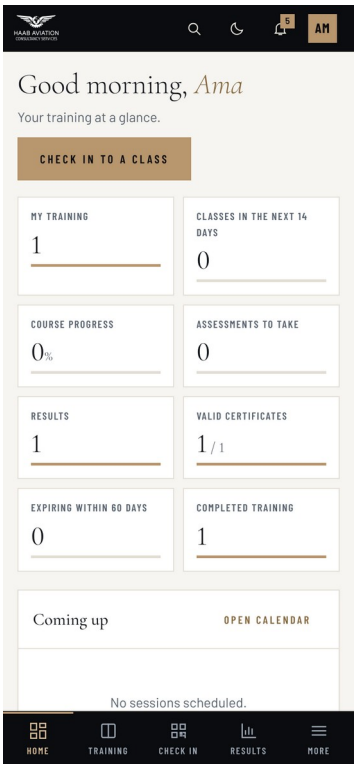
- **Sidebar** (248px, dark): logo, reporting-period line, then module groups that expand to their pages. The current page has a gold bar. Items a role cannot use are not rendered.
- **Top bar:** menu toggle, global search, notifications bell with count, day/night toggle, account menu.
- **Content:** page title with a one-line description and the page's main action on the right, then cards, tiles and tables on the surface colour. Maximum comfortable reading width; tables scroll inside their card.
- **Breakpoints:** 1100px (sidebar collapses to a drawer), 900px (two-column grids stack), 700px (phone layout).



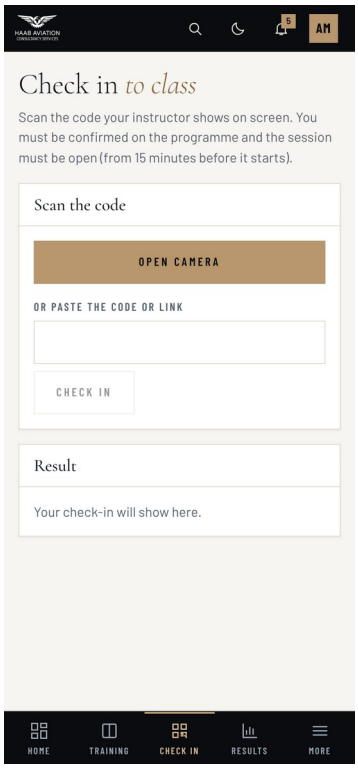
Administrator home: tiles, charts and the To do list

Phone

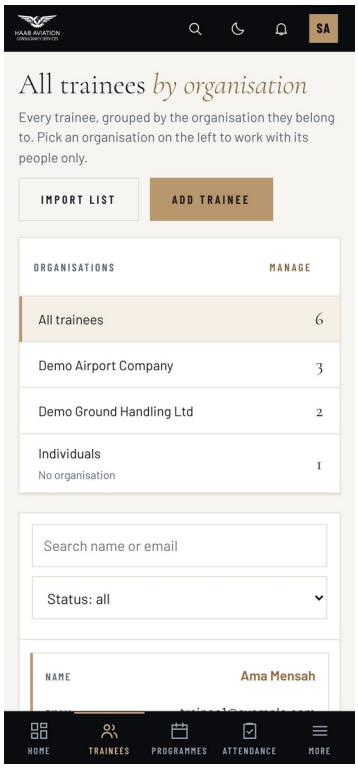
- A **bottom bar** with the four things each role does most, plus **More**, which opens the full menu.
- Tables become **cards**; filters and forms open as **sheets** from the bottom; buttons grow for touch.
- The sign-in page, check-in, assessments, results and certificates were designed for the phone first.



Trainee home



Check in



Trainees as cards

4.4 Component library

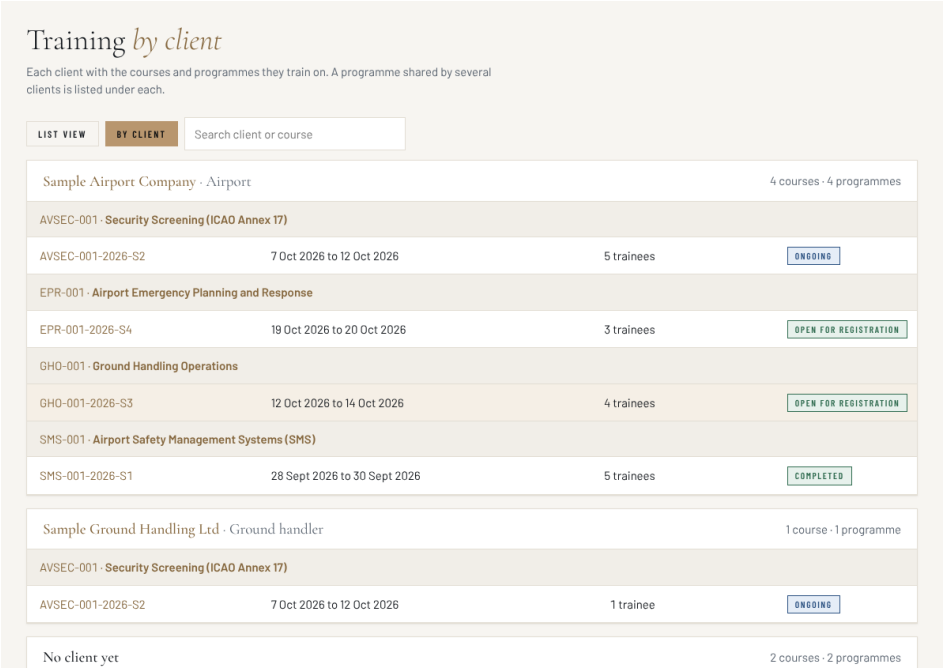
Component	Behaviour and rules
Page head	Title (serif with italic accent word), one-line description, primary action at the right, optional breadcrumbs.
Data list	Search box, filters as selects, columns with optional links, row actions, paging, an empty state with a hint, a loading state and an inline error note. Every list in the product uses it.
Cards and tiles	Card with serif title; KPI tile shows a big lining number and a progress bar against a meaningful total.
Badges	Outlined one-pixel tags in ok, warn, danger, info and neutral tones, with text in capitals. Colour is never the only signal.
Forms	Labelled fields, help text under the field, required marks, inline validation messages from the server, modal forms for create and edit, field types: text, number, date, datetime, select, checkbox, tags, textarea, url, email, password.
Confirm dialog	Destructive or irreversible actions: title, a sentence saying what will happen and what will not, a red confirm button. Delete dialogs add what goes with the record and a note that the official record is protected.
Toasts	Short confirmation ("Saved", "Deleted") at the corner; errors show the server's message.
Tabs	Gold underline on the current tab with optional counts, used on course and programme pages.
To do card	Role-specific list of what needs doing with counts and links; shown first on every home page.
Close-out checklist	On a programme: the steps from start to release with the single next step highlighted and a button for it.
Charts	Bars, lines and half-donuts from a small categorical palette checked in both themes, with direct labels or legends.
Empty, loading, error	Every data view has all three. Empty states state the next step; errors say what to try.

4.5 Key screens

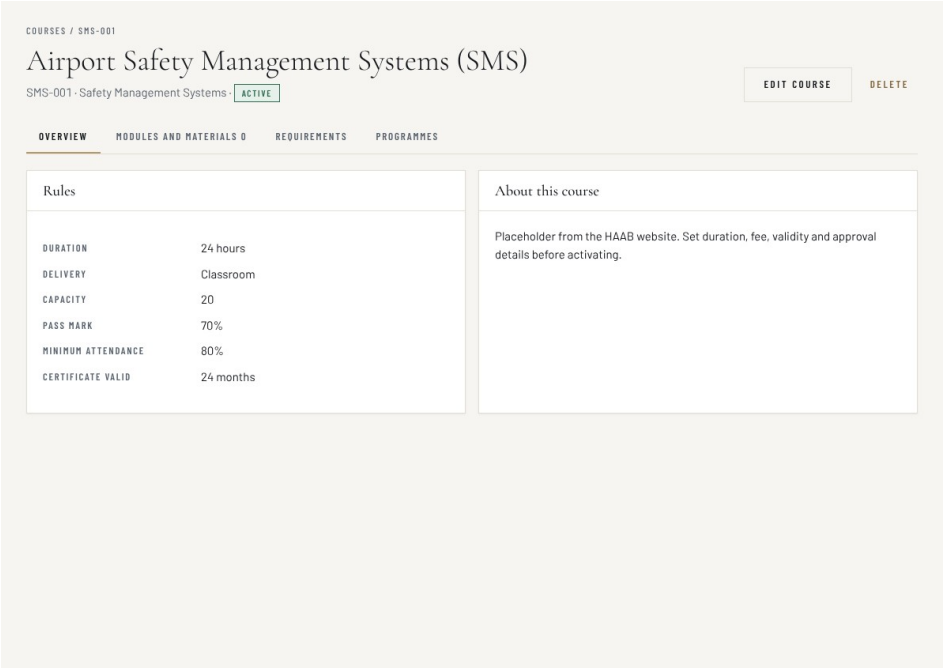
Each screen below is described by its job, its main content and its main action.

Screen	Job	Main content	Main action
Sign in	Get in simply	Logo, email, password, forgot link, request-an-account link when allowed, day/night toggle	Sign in
Home (per role)	Start the day	To do list, KPI tiles, charts or upcoming classes	The first To do item
Courses	Know what HAAB teaches	Catalogue list (fee column only when switched on), course page with Rules, Modules and materials, Requirements, Programmes tabs	New course / Add material

Screen	Job	Main content	Main action
Programmes	Run a programme	List with places and status, By client view, programme page with Overview, Sessions, Registrations, Attendance, Assessments, Results and the close-out checklist	Open registration / next close-out step
Attendance session	Record who came	Roster with status, QR panel with large view, mark-all-present	Save attendance
Registrations	Control places	Pending, confirmed, waitlisted; payment state; confirm and cancel	Confirm
Assessment builder and exam	Test competence	Question picker, settings, timed exam with countdown and saved-time note	Publish / Submit
Marking queue	Mark written answers	Attempts needing marking, marked and all; marks and feedback per answer	Save marks
Results	Decide and release	Result per trainee with attendance and scores, pending reason, finalise, override, release	Finalise / Release results
Certificates	Prove competence	Register with status and expiry, PDF, verify link, reissue, renew, revoke	Issue / Download
Finance	Get paid	Invoices with status, invoice page with payments, receipts, refunds	Record payment
Settings	Control the platform	Pages from the settings registry, Users, Roles, Menu Access matrix, integrations	Save
Verify (public)	Check a certificate	Validity badge, name, course, dates only	None



Programmes grouped by client



A course page (fee hidden by default)

4.6 Accessibility

- Every form control has a visible label or an accessible name; help text is tied to its field.
- A visible two-pixel gold focus ring on every interactive element; the page is fully keyboard operable.
- Colour contrast meets WCAG AA in both themes (gold as text uses the darker `--accent-text` on white).
- Status is always conveyed by text as well as colour.
- Motion is reduced when the device asks for it; pages have no auto-playing animation.
- Tables keep real table semantics on desktop; on phones the card layout keeps reading order.
- Buttons and inputs are enlarged for touch on phones.

4.7 Content and microcopy

- Buttons start with a verb: Save, Add module, Open registration, Mark everyone present.
- Refusals say **why** and **what to do instead**: "Archive it instead" or "Revoke the certificate or suspend the account".
- Empty states say what the page is for and the first step.
- Use "registration" for a place on a programme, "trainee" for a learner, "client" for an organisation, "programme" for a scheduled run of a course.
- Dates read like "7 Oct 2026". Money reads like "GHS 1,500.00". Percentages have no decimals unless needed.

4.8 Design QA checklist

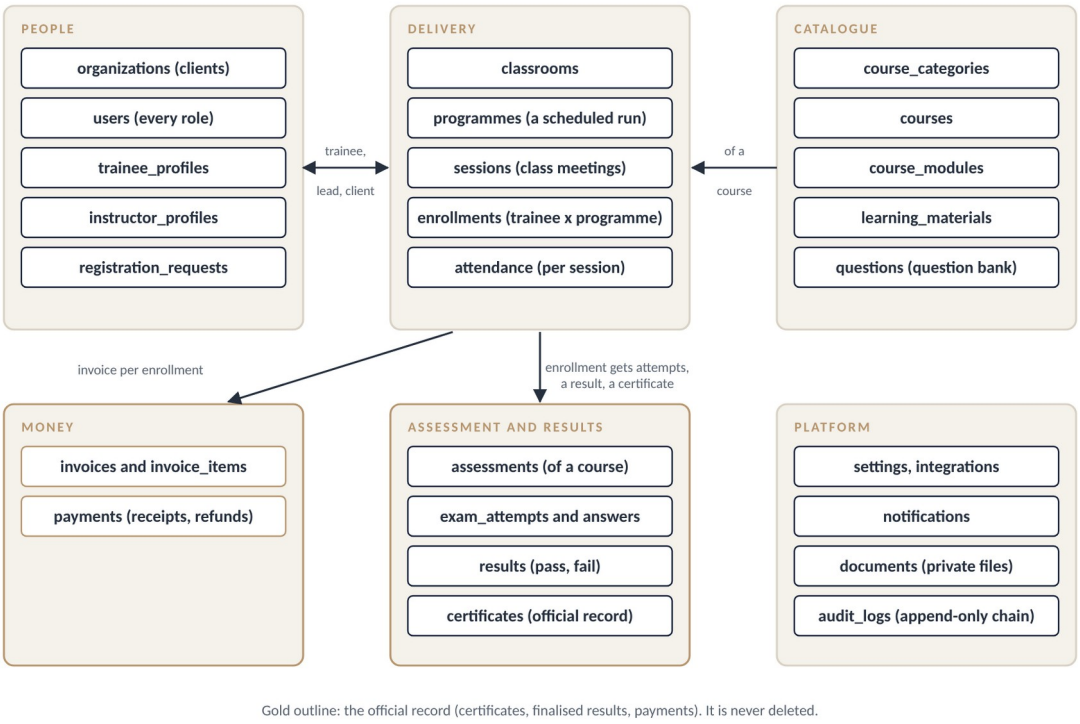
Check	Done when
Both themes	Every new screen is viewed in Day and Night with no raw colours.
Three states	Empty, loading and error states exist.

Check	Done when
Every role	The screen is opened as each role that can reach it and shows only that role's data and fee rules.
Phone	Checked at 390px wide: no sideways scroll and touch-sized buttons.
Keyboard	Tab order is sensible and focus is visible.
Destructive actions	Each one asks first, says what goes with it, and shows the server's reason when refused.

Part 5. Backend Schema

5.1 Overview

The database is **PostgreSQL 16** (Supabase). It holds **38 tables** created by ten migrations. Every table has row-level security enabled with no policies, so only the API's own connection can read or write. Identifiers are UUIDs generated by the database (the audit log uses a bigint sequence); times are `timestamptz`; dates are `date`; money is `numeric`.



Data model: the main groups of tables

Design rules

- **One users table.** Role is a column with a check constraint. A check also guarantees that only trainees and client administrators can belong to an organisation.
- **Status columns are check-constrained** to a fixed list (see 5.8), so no code path can invent a state.
- **The database refuses double booking.** Exclusion constraints (btree_gist) on sessions prevent one instructor or one classroom being in two places at once.
- **Uniqueness protects the record:** one live certificate per registration; one registration per trainee per programme; case-insensitive unique names for organisations and classrooms; unique certificate numbers and verification tokens.
- **Official record:** certificates, finalised results and payments have no delete path in the application. Foreign keys that would remove them are not cascaded.
- **Soft pointers** such as `created_by` or `registered_by` are nullable, so a person can be deleted without losing the record they touched.
- **Audit log** rows are chained by a trigger and cannot be updated, deleted or truncated.

- **Sequences are tables** (`certificate_sequences`, `invoice_sequences`, `receipt_sequences`) incremented inside the issuing transaction, so numbers never repeat or skip because of a rollback.

5.2 People and organisations

organizations

Client organisations (airlines, airports, ground handlers and others) whose staff train with HAAB.

Column	Type	Rules
`id`	uuid	Primary key
`name`	text	Required
`type`	text	Required Default other
`contact_name`	text	
`contact_email`	text	
`contact_phone`	text	
`address`	text	
`billing_email`	text	
`billing_address`	text	
`status`	text	Required Default active
`created_at`	timestamptz	Required Default now()
`updated_at`	timestamptz	Required Default now()

Constraints and indexes

- Check: status = ANY (ARRAY['active', 'inactive'])
- Check: type = ANY (ARRAY['airline', 'airport', 'ground_handler', 'cargo_operator', 'security_organization', 'other'])
- Unique index `organizations_name_key` (lower(name))

users

Every person who can sign in, in one table, with a single role. Trainees and client administrators may belong to one organisation; HAAB staff belong to none.

Column	Type	Rules
`id`	uuid	Primary key
`auth_user_id`	uuid	Unique
`email`	text	Required
`full_name`	text	Required
`phone`	text	
`role`	text	Required
`organization_id`	uuid	→ organizations
`status`	text	Required Default invited
`last_login_at`	timestamptz	
`created_by`	uuid	→ users

Column	Type	Rules
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()
mfa_enrolled	boolean	Required Default false
can_finalise_results	boolean	Required Default false

Constraints and indexes

- Check: (role <> 'org_admin') OR (organization_id IS NOT NULL)
- Check: (role = ANY (ARRAY['trainee', 'org_admin'])) OR (organization_id IS NULL)
- Check: role = ANY (ARRAY['super_admin', 'training_admin', 'instructor', 'trainee', 'org_admin', 'finance_officer', 'auditor'])
- Check: status = ANY (ARRAY['invited', 'active', 'suspended'])
- Index users_name_trgm (full_name gin_trgm_ops)
- Index users_org_idx (organization_id)
- Index users_role_idx (role)
- Unique index users_auth_user_id_key (auth_user_id)
- Unique index users_email_key (lower(email))

trainee_profiles

Extra details about a trainee: contact, employer, aviation role, encrypted national ID number.

Column	Type	Rules
user_id	uuid	Primary key → users (cascade)
employer	text	
aviation_role	text	
licence_number	text	
licence_expiry	date	
nationality	text	
date_of_birth	date	
id_type	text	
id_number_enc	text	
qualifications	text	
emergency_contact	text	
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

instructor_profiles

Extra details about an instructor: employment type, specialties, accreditation and its expiry.

Column	Type	Rules
user_id	uuid	Primary key → users (cascade)

Column	Type	Rules
employment_type	text	Required Default associate
qualifications	text	
specialties	text	
accreditation_body	text	
accreditation_expiry	date	
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: employment_type = ANY (ARRAY['employee', 'associate'])

registration_requests

Requests for an account made from the sign-in page, waiting for approval.

Column	Type	Rules
id	uuid	Primary key
email	text	Required
full_name	text	Required
phone	text	
organisation_text	text	
status	text	Required Default pending
decided_by	uuid	→ users
decided_at	timestamptz	
reject_reason	text	
user_id	uuid	→ users
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: status = ANY (ARRAY['pending', 'approved', 'rejected'])
- Index registration_status_idx (status, created_at DESC)
- Unique index registration_pending_email (lower(email)) where (status = 'pending')

5.3 Catalogue and learning material

course_categories

Groups of courses. The category code appears in certificate numbers.

Column	Type	Rules
id	uuid	Primary key
code	text	Required
name	text	Required
description	text	

Column	Type	Rules
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: code ~ '^[A-Z]{2,5}$'
- Unique index course_categories_code_key (code)
- Unique index course_categories_name_key (lower(name))

courses

The catalogue: what HAAB teaches, with rules for pass mark, attendance, validity and (hidden) fee and recognition.

Column	Type	Rules
id	uuid	Primary key
code	text	Required
title	text	Required
category_id	uuid	→ course_categories Required
description	text	
objectives	text	
duration_hours	numeric(6,1)	Required Default 0
delivery_method	text	Required Default classroom
target_audience	text	
capacity	integer	Required Default 20
fee	numeric(12,2)	Required Default 0
currency	text	Required Default GHS
pass_mark	integer	Required Default 70
min_attendance_pct	integer	Required Default 80
validity_months	integer	
approving_body	text	
approval_reference	text	
approval_expiry	date	
status	text	Required Default draft
created_by	uuid	→ users
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: (min_attendance_pct >= 0) AND (min_attendance_pct <= 100)

- Check: (pass_mark >= 0) AND (pass_mark <= 100)
- Check: (validity_months IS NULL) OR (validity_months > 0)
- Check: capacity > 0
- Check: delivery_method = ANY (ARRAY['classroom', 'online', 'blended', 'practical'])
- Check: duration_hours >= (0)
- Check: fee >= (0)
- Check: status = ANY (ARRAY['draft', 'active', 'archived'])
- Index `courses_category_idx` (category_id)
- Index `courses_title_trgm` (title gin_trgm_ops)
- Unique index `courses_code_key` (upper(code))

course_prerequisites

Which courses must be completed before another.

Column	Type	Rules
`course_id`	uuid	Primary key → courses (cascade)
`prerequisite_id`	uuid	Primary key → courses

Constraints and indexes

- Check: course_id <> prerequisite_id

course_instructors

Optional list of instructors who teach a course (empty means any instructor of its programmes).

Column	Type	Rules
`course_id`	uuid	Primary key → courses (cascade)
`instructor_id`	uuid	Primary key → users

course_modules

Topics within a course, shown on the certificate.

Column	Type	Rules
`id`	uuid	Primary key
`course_id`	uuid	→ courses (cascade) Required
`title`	text	Required
`description`	text	
`position`	integer	Required Default 0
`created_at`	timestamptz	Required Default now()
`created_by`	uuid	→ users

Constraints and indexes

- Index `course_modules_course_idx` (course_id, "position")

learning_materials

Links and uploaded files inside a module, with who added each.

Column	Type	Rules
id	uuid	Primary key
module_id	uuid	→ course_modules (cascade) Required
title	text	Required
kind	text	Required
url	text	
document_id	uuid	→ documents
required	boolean	Required Default true
position	integer	Required Default 0
created_at	timestamptz	Required Default now()
created_by	uuid	→ users

Constraints and indexes

- Check: (url IS NOT NULL) OR (document_id IS NOT NULL)
- Check: kind = ANY (ARRAY['video', 'pdf', 'presentation', 'document', 'audio', 'image', 'scorm', 'link'])
- Index learning_materials_module_idx (module_id, "position")

material_progress

Each trainee's progress through a learning material.

Column	Type	Rules
trainee_id	uuid	Primary key → users (cascade)
material_id	uuid	Primary key → learning_materials (cascade)
status	text	Required Default not_started
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: status = ANY (ARRAY['not_started', 'in_progress', 'completed'])

questions

The question bank, by course, topic, difficulty and type.

Column	Type	Rules
id	uuid	Primary key
course_id	uuid	→ courses Required
module_id	uuid	→ course_modules
topic	text	

Column	Type	Rules
difficulty	text	Required Default medium
type	text	Required
prompt	text	Required
options	jsonb	
answer	jsonb	
marks	numeric(6,2)	Required Default 1
explanation	text	
status	text	Required Default active
created_by	uuid	→ users
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: difficulty = ANY (ARRAY['easy', 'medium', 'hard'])
- Check: marks > (0)
- Check: status = ANY (ARRAY['active', 'retired'])
- Check: type = ANY (ARRAY['mcq_single', 'mcq_multi', 'true_false', 'short_answer', 'essay', 'matching', 'scenario'])
- Index questions_course_idx (course_id, status)

5.4 Programmes and delivery

classrooms

Rooms with capacity, used to detect double booking.

Column	Type	Rules
id	uuid	Primary key
name	text	Required
location	text	
capacity	integer	Required Default 20
active	boolean	Required Default true
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: capacity > 0
- Unique index classrooms_name_key (lower(name))

programmes

One scheduled run of a course: dates, venue, lead instructor, capacity, fee, optional reservation for one client and a status.

Column	Type	Rules
id	uuid	Primary key
code	text	Required
course_id	uuid	→ courses Required
title	text	Required
start_date	date	Required
end_date	date	Required
location	text	
classroom_id	uuid	→ classrooms
delivery_method	text	Required Default classroom
lead_instructor_id	uuid	→ users
capacity	integer	Required
registration_deadline	date	
fee	numeric(12,2)	Required Default 0
currency	text	Required Default GHS
organization_id	uuid	→ organizations
status	text	Required Default draft
created_by	uuid	→ users
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: capacity > 0
- Check: delivery_method = ANY (ARRAY['classroom', 'online', 'blended', 'practical'])
- Check: end_date >= start_date
- Check: fee >= (0)
- Check: status = ANY (ARRAY['draft', 'open_for_registration', 'registration_closed', 'ongoing', 'completed', 'cancelled'])
- Index programmes_course_idx (course_id)
- Index programmes_dates_idx (start_date, end_date)
- Index programmes_status_idx (status)
- Unique index programmes_code_key (upper(code))

sessions

Individual class meetings within a programme.

Column	Type	Rules
id	uuid	Primary key
programme_id	uuid	→ programmes (cascade) Required

Column	Type	Rules
`title`	text	Required
`kind`	text	Required Default class
`starts_at`	timestamptz	Required
`ends_at`	timestamptz	Required
`instructor_id`	uuid	→ users
`classroom_id`	uuid	→ classrooms
`location`	text	
`qr_secret`	text	Required Default replace(((gen_random_uuid())|| (gen_random_uuid())), '-', '')
`created_at`	timestamptz	Required Default now()
`updated_at`	timestamptz	Required Default now()

Constraints and indexes

- Check: ends_at > starts_at
- Check: kind = ANY (ARRAY['class', 'exam', 'practical'])
- EXCLUDE USING gist (classroom_id WITH =, tstzrange(starts_at, ends_at) WITH &&) WHERE ((classroom_id IS NOT NULL))
- EXCLUDE USING gist (instructor_id WITH =, tstzrange(starts_at, ends_at) WITH &&) WHERE ((instructor_id IS NOT NULL))
- Index `sessions_no_instructor_overlap` (instructor_id, tstzrange(starts_at, ends_at)) where (instructor_id IS NOT NULL)
- Index `sessions_no_room_overlap` (classroom_id, tstzrange(starts_at, ends_at)) where (classroom_id IS NOT NULL)
- Index `sessions_programme_idx` (programme_id, starts_at)
- Index `sessions_time_idx` (starts_at)

enrollments

A trainee's place on a programme with its status and sponsoring organisation.

Column	Type	Rules
`id`	uuid	Primary key
`programme_id`	uuid	→ programmes Required
`trainee_id`	uuid	→ users Required
`status`	text	Required Default pending
`sponsor_organization_id`	uuid	→ organizations
`registered_by`	uuid	→ users
`confirmed_by`	uuid	→ users
`confirmed_at`	timestamptz	
`cancel_reason`	text	
`created_at`	timestamptz	Required Default now()
`updated_at`	timestamptz	Required Default now()

Constraints and indexes

- Check: status = ANY (ARRAY['pending', 'confirmed', 'waitlisted', 'cancelled', 'completed', 'no_show'])

- UNIQUE (programme_id, trainee_id)
- Index `enrollments_status_idx` (programme_id, status)
- Index `enrollments_trainee_idx` (trainee_id)
- Unique index `enrollments_programme_id_trainee_id_key` (programme_id, trainee_id)

attendance

Whether a trainee was present at a session, and how it was recorded.

Column	Type	Rules
`id`	uuid	Primary key
`session_id`	uuid	→ sessions (cascade) Required
`trainee_id`	uuid	→ users Required
`status`	text	Required
`method`	text	Required Default manual
`marked_by`	uuid	→ users
`marked_at`	timestamptz	Required Default now()
`note`	text	
`latitude`	numeric(9,6)	
`longitude`	numeric(9,6)	

Constraints and indexes

- Check: method = ANY (ARRAY['manual', 'qr'])
- Check: status = ANY (ARRAY['present', 'absent', 'late', 'excused'])
- UNIQUE (session_id, trainee_id)
- Index `attendance_trainee_idx` (trainee_id)
- Unique index `attendance_session_id_trainee_id_key` (session_id, trainee_id)

5.5 Assessment, results and certificates

assessments

Exams, quizzes and practicals set for a course or programme.

Column	Type	Rules
`id`	uuid	Primary key
`course_id`	uuid	→ courses Required
`programme_id`	uuid	→ programmes
`title`	text	Required
`kind`	text	Required Default exam
`duration_minutes`	integer	Required Default 60
`pass_mark`	integer	Required Default 70

Column	Type	Rules
max_attempts	integer	Required Default 1
randomize	boolean	Required Default true
release_mode	text	Required Default manual
opens_at	timestamptz	
closes_at	timestamptz	
status	text	Required Default draft
created_by	uuid	→ users
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: (pass_mark >= 0) AND (pass_mark <= 100)
- Check: duration_minutes > 0
- Check: kind = ANY (ARRAY['exam', 'quiz', 'practical'])
- Check: max_attempts > 0
- Check: release_mode = ANY (ARRAY['immediate', 'manual'])
- Check: status = ANY (ARRAY['draft', 'published', 'closed'])
- Index assessments_programme_idx (programme_id)

assessment_questions

Which questions an assessment contains, in order, with marks.

Column	Type	Rules
assessment_id	uuid	Primary key → assessments (cascade)
question_id	uuid	Primary key → questions
position	integer	Required Default 0
marks	numeric(6,2)	Required

Constraints and indexes

- Check: marks > (0)

exam_attempts

One sitting of an assessment by a trainee, with times, score and status.

Column	Type	Rules
id	uuid	Primary key
assessment_id	uuid	→ assessments Required
trainee_id	uuid	→ users Required
attempt_no	integer	Required

Column	Type	Rules
started_at	timestamptz	Required Default now()
deadline_at	timestamptz	Required
submitted_at	timestamptz	
status	text	Required Default in_progress
question_order	jsonb	Required
score	numeric(8,2)	
max_score	numeric(8,2)	
percentage	numeric(5,2)	
results_released	boolean	Required Default false
marked_by	uuid	→ users
marked_at	timestamptz	

Constraints and indexes

- Check: status = ANY (ARRAY['in_progress', 'submitted', 'marked'])
- UNIQUE (assessment_id, trainee_id, attempt_no)
- Index exam_attempts_trainee_idx (trainee_id)
- Unique index exam_attempts_assessment_id_trainee_id_attempt_no_key (assessment_id, trainee_id, attempt_no)

attempt_answers

The answer to each question in an attempt, with automatic or manual marks and feedback.

Column	Type	Rules
attempt_id	uuid	Primary key → exam_attempts (cascade)
question_id	uuid	Primary key → questions
response	jsonb	
awarded_marks	numeric(6,2)	
needs_manual	boolean	Required Default false
feedback	text	
marked_by	uuid	→ users
saved_at	timestamptz	Required Default now()

results

The outcome for one registration: status, score, attendance, finalisation and release. Part of the official record once finalised.

Column	Type	Rules
id	uuid	Primary key
enrollment_id	uuid	→ enrollments Unique Required

Column	Type	Rules
final_score	numeric(5,2)	
attendance_pct	numeric(5,2)	
status	text	Required Default pending
finalised	boolean	Required Default false
finalised_by	uuid	→ users
finalised_at	timestamptz	
released	boolean	Required Default false
remarks	text	
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: status = ANY (ARRAY['pass', 'fail', 'pending', 'absent', 'disqualified'])
- Unique index results_enrollment_id_key (enrollment_id)

result_changes

History of overrides of a result, with reason and who made them.

Column	Type	Rules
id	uuid	Primary key
result_id	uuid	→ results Required
old_status	text	
new_status	text	
old_score	numeric(5,2)	
new_score	numeric(5,2)	
reason	text	Required
changed_by	uuid	→ users Required
changed_at	timestamptz	Required Default now()

certificate_templates

Certificate designs: signatory, title and accent colour.

Column	Type	Rules
id	uuid	Primary key
name	text	Required
organization_id	uuid	→ organizations
heading	text	Required Default Certificate of Completion
signatory_name	text	

Column	Type	Rules
signatory_title	text	
footer_text	text	
accent_color	text	Required Default #b8966e
is_default	boolean	Required Default false
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: accent_color ~ '^#[0-9a-fA-F]{6}$'
- Unique index certificate_templates_one_default (is_default) where is_default

certificate_sequences

Counters for certificate serial numbers, one per year and category.

Column	Type	Rules
year	integer	Primary key
category_code	text	Primary key
last_value	integer	Required Default 0

certificates

Issued certificates with number, verification token, dates and status. Official record.

Column	Type	Rules
id	uuid	Primary key
number	text	Unique Required
verification_token	text	Unique Required
enrollment_id	uuid	→ enrollments Required
trainee_id	uuid	→ users Required
course_id	uuid	→ courses Required
programme_id	uuid	→ programmes Required
template_id	uuid	→ certificate_templates
issued_at	date	Required Default CURRENT_DATE
expires_at	date	
status	text	Required Default valid
revoked_at	timestamptz	
revoked_by	uuid	→ users
revoke_reason	text	

Column	Type	Rules
renewal_of	uuid	→ certificates
issued_by	uuid	→ users
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: (status <> 'revoked') OR ((revoked_at IS NOT NULL) AND (revoke_reason IS NOT NULL))
- Check: status = ANY (ARRAY['valid', 'revoked'])
- Index certificates_expiry_idx (expires_at) where (status = 'valid')
- Index certificates_trainee_idx (trainee_id)
- Unique index certificates_number_key (number)
- Unique index certificates_one_live_per_enrollment (enrollment_id) where (status = 'valid')
- Unique index certificates_verification_token_key (verification_token)

5.6 Finance

invoice_sequences

Counter for invoice numbers by year.

Column	Type	Rules
year	integer	Primary key
last_value	integer	Required Default 0

invoices

Invoices to an organisation or an individual for a registration or other work.

Column	Type	Rules
id	uuid	Primary key
number	text	Unique Required
organization_id	uuid	→ organizations
trainee_id	uuid	→ users
programme_id	uuid	→ programmes
enrollment_id	uuid	→ enrollments
issue_date	date	Required Default CURRENT_DATE
due_date	date	
currency	text	Required Default GHS
subtotal	numeric(12,2)	Required Default 0
tax_rate	numeric(5,2)	Required Default 0
tax_amount	numeric(12,2)	Required Default 0
total	numeric(12,2)	Required Default 0

Column	Type	Rules
amount_paid	numeric(12,2)	Required Default 0
status	text	Required Default pending
purchase_order	text	
notes	text	
created_by	uuid	→ users
created_at	timestamptz	Required Default now()
updated_at	timestamptz	Required Default now()

Constraints and indexes

- Check: (organization_id IS NOT NULL) OR (trainee_id IS NOT NULL)
- Check: status = ANY (ARRAY['pending', 'partially_paid', 'paid', 'failed', 'refunded', 'cancelled'])
- Index invoices_org_idx (organization_id)
- Index invoices_status_idx (status)
- Index invoices_trainee_idx (trainee_id)
- Unique index invoices_number_key (number)

invoice_items

Lines on an invoice.

Column	Type	Rules
id	uuid	Primary key
invoice_id	uuid	→ invoices (cascade) Required
description	text	Required
quantity	numeric(8,2)	Required Default 1
unit_price	numeric(12,2)	Required

Constraints and indexes

- Check: quantity > (0)
- Check: unit_price >= (0)

receipt_sequences

Counter for receipt numbers by year.

Column	Type	Rules
year	integer	Primary key
last_value	integer	Required Default 0

payments

Money received, refunded or failed against an invoice. Official record.

Column	Type	Rules
id	uuid	Primary key

Column	Type	Rules
invoice_id	uuid	→ invoices Required
receipt_number	text	Unique Required
amount	numeric(12,2)	Required
method	text	Required
reference	text	
received_at	timestamptz	Required Default now()
status	text	Required Default paid
note	text	
recorded_by	uuid	→ users Required
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: amount > (0)
- Check: method = ANY (ARRAY['cash', 'bank_transfer', 'mobile_money', 'card', 'cheque', 'other'])
- Check: status = ANY (ARRAY['paid', 'failed', 'refunded', 'cancelled'])
- Index payments_invoice_idx (invoice_id)
- Unique index payments_receipt_number_key (receipt_number)

5.7 Compliance, documents and platform

compliance_requirements

Mandatory training rules by course, client and aviation role, with grace periods.

Column	Type	Rules
id	uuid	Primary key
course_id	uuid	→ courses Required
organization_id	uuid	→ organizations
aviation_role	text	
grace_days	integer	Required Default 0
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: grace_days >= 0
- Unique index compliance_requirements_key (course_id, COALESCE(organization_id, '00000000-0000-0000-0000-000000000000'), COALESCE(aviation_role, ''))

documents

Metadata for uploaded files (the file itself is in private storage).

Column	Type	Rules
id	uuid	Primary key
category	text	Required Default other
filename	text	Required
mime_type	text	Required
size_bytes	bigint	Required
sha256	text	Required
storage_key	text	Required
owner_user_id	uuid	→ users
organization_id	uuid	→ organizations
confidential	boolean	Required Default true
uploaded_by	uuid	→ users
created_at	timestamptz	Required Default now()

Constraints and indexes

- Check: category = ANY (ARRAY['identification', 'certificate', 'licence', 'training_record', 'course_material', 'attendance', 'examination', 'instructor_qualification', 'financial', 'other'])
- Check: size_bytes >= 0
- Index documents_org_idx (organization_id)
- Index documents_owner_idx (owner_user_id)

notifications

In-app notices and the email outbox.

Column	Type	Rules
id	uuid	Primary key
user_id	uuid	→ users (cascade)
to_email	text	
channel	text	Required Default in_app
kind	text	Required
subject	text	Required
body	text	Required
status	text	Required Default pending
attempts	integer	Required Default 0
error	text	
dedupe_key	text	
created_at	timestamptz	Required Default now()
sent_at	timestamptz	

Constraints and indexes

- Check: channel = ANY (ARRAY['in_app', 'email', 'sms'])
- Check: status = ANY (ARRAY['pending', 'sent', 'failed', 'read'])
- Index `notifications_pending_idx` (status) where (status = 'pending')
- Index `notifications_user_idx` (user_id, created_at DESC)
- Unique index `notifications_dedupe` (dedupe_key) where (dedupe_key IS NOT NULL)

settings

Administrative settings as JSON, one row per page key.

Column	Type	Rules
`key`	text	Primary key
`value`	jsonb	Required
`updated_by`	uuid	→ users
`updated_at`	timestamptz	Required Default now()

integrations

Connections to outside services; secret values are stored encrypted and never returned.

Column	Type	Rules
`key`	text	Primary key
`enabled`	boolean	Required Default false
`config`	jsonb	Required Default {}
`secrets`	jsonb	Required Default {}
`secrets_updated_at`	timestamptz	
`updated_by`	uuid	→ users
`updated_at`	timestamptz	Required Default now()

audit_logs

Append-only, hash-chained record of significant actions.

Column	Type	Rules
`id`	bigint	Primary key
`at`	timestamptz	Required Default now()
`actor_id`	uuid	
`actor_email`	text	
`actor_role`	text	
`action`	text	Required
`entity_type`	text	Required
`entity_id`	text	
`ip`	text	
`before_data`	jsonb	

Column	Type	Rules
after_data	jsonb	
seq	bigint	Required Default 0
prev_hash	text	Required Default
hash	text	Required Default

Constraints and indexes

- Index audit_logs_actor_idx (actor_id)
- Index audit_logs_at_idx (at DESC)
- Index audit_logs_entity_idx (entity_type, entity_id)
- Unique index audit_logs_seq_key (seq)

5.8 Allowed values

Each of these columns is restricted by a check constraint.

Column	Allowed values
users.role	super_admin, training_admin, instructor, trainee, org_admin, finance_officer, auditor
users.status	invited, active, suspended
organizations.type	airline, airport, ground_handler, cargo_operator, security_organization, other
organizations.status	active, inactive
instructor_profiles.employment_type	employee, associate
courses.status	draft, active, archived
courses.delivery_method, programmes.delivery_method	classroom, online, blended, practical
programmes.status	draft, open_for_registration, registration_closed, ongoing, completed, cancelled
sessions.kind	class, exam, practical
enrollments.status	pending, confirmed, waitlisted, cancelled, completed, no_show
attendance.status	present, absent, late, excused
attendance.method	manual, qr
learning_materials.kind	video, pdf, presentation, document, audio, image, scorm, link
material_progress.status	not_started, in_progress, completed
questions.type	mcq_single, mcq_multi, true_false, short_answer, essay, matching, scenario
questions.difficulty	easy, medium, hard
questions.status	active, retired
assessments.kind	exam, quiz, practical
assessments.status	draft, published, closed
assessments.release_mode	immediate, manual
exam_attempts.status	in_progress, submitted, marked
results.status	pass, fail, pending, absent, disqualified
certificates.status	valid, revoked

Column	Allowed values
`invoices.status`	pending, partially_paid, paid, failed, refunded, cancelled
`payments.status`	paid, failed, refunded, cancelled
`payments.method`	cash, bank_transfer, mobile_money, card, cheque, other
`documents.category`	identification, certificate, licence, training_record, course_material, attendance, examination, instructor_qualification, financial, other
`notifications.channel`	in_app, email, sms
`notifications.status`	pending, sent, failed, read
`registration_requests.status`	pending, approved, rejected

5.9 Functions, triggers and extensions

Object	Purpose
`set_updated_at()` with `*_updated` triggers	Keeps `updated_at` current on the main tables.
`audit_chain()` with `audit_logs_chain`	Before each insert: takes a lock, assigns the next sequence number and computes the SHA-256 hash linking to the previous row.
`audit_immutable()` with `audit_logs_no_update` and `audit_logs_no_truncate`	Refuses any update, delete or truncate of the audit log.
`audit_verify()`	Walks the chain and returns the first broken link, if any.
Extensions `btree_gist`, `pg_trgm`	Exclusion constraints for double booking; trigram indexes for fast search.
Row-level security on every table	Enabled with no policies: deny by default for the Supabase auto-generated REST interface.

5.10 Migrations

File	Change
`0001_core.sql`	All core tables, constraints, indexes, audit chain, triggers, default settings and row-level security.
`0002_integrations.sql`	Integrations table for encrypted outside-service credentials.
`0003_audit_verify.sql`	The `audit_verify()` function.
`0004_staff_have_no_organisation.sql`	Only trainees and client administrators can belong to an organisation.
`0005_optional_mfa.sql`	Two-step sign-in becomes optional and policy-driven.
`0006_registration_requests.sql`	Account requests from the sign-in page.
`0007_instructor_finalise.sql`	`users.can_finalise_results` for the instructor right.
`0008_rotate_qr_secrets.sql`	Rotates every session's QR secret.

File	Change
`0009_clean_material_links.sql`	Removes learning links that are not http or https.
`0010_material_authors.sql`	`created_by` on modules and materials for instructor ownership.

5.11 Retention and deletion

Data	Rule
Certificates, finalised results, recorded payments	Never deleted. Revoke, refund or suspend instead.
Audit log	Never edited or deleted by the platform. Only the reset command with an explicit option can clear it, and it then records that it did.
Demo and test data	Deletable one record at a time by a Super Administrator, or in bulk with the reset command.
Deleted people	Their registrations, attendance, attempts and unpaid invoices go with them; sign-in account removed; soft author pointers released; a copy of what was removed is kept in the audit log.
Files	Removed from storage when their document is deleted or the platform is reset.
Backups	Provided by the database host; restore drills are recorded in settings.